

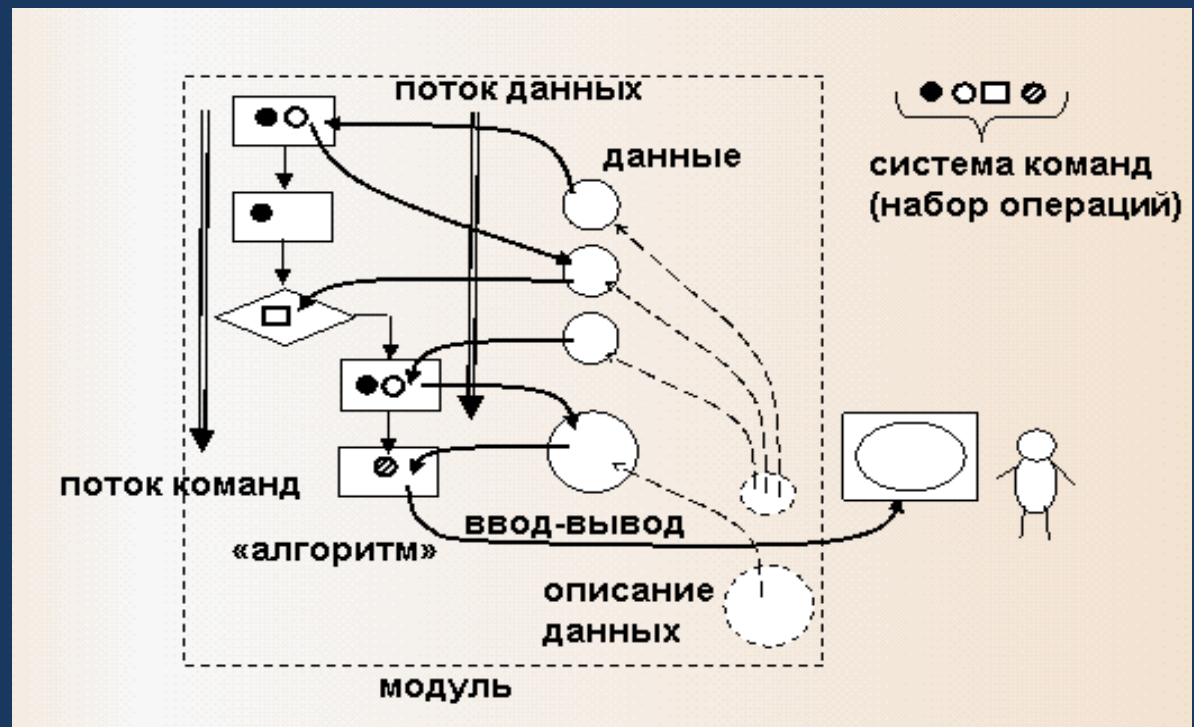


Процессы, потоки, прерывания

Факты из компьютерной архитектуры

Архитектура фон Неймана (1940-е годы):

- прямо адресуемая память
- принцип хранимой программы: программа с точки зрения представления в памяти = данные
- **поток управления (поток команд)** – последовательность команд с адресами операндов

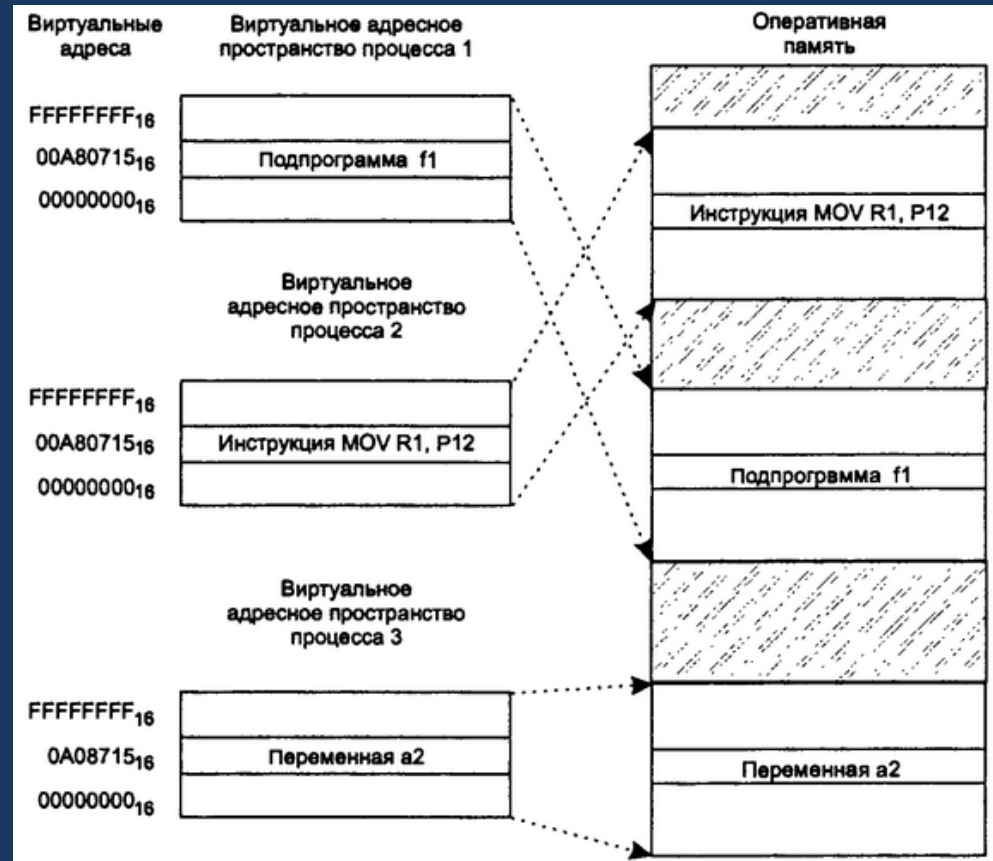




Факты из компьютерной архитектуры

Прямо – адресуемая память

- **машинное слово (МС)** – байт (byte), короткое МС (short), стандартное МС = разрядность процессора, длинное (long)
- **регистры** – собственные МС процессора
- **физическое АП (ФАП)** –
АП реальных адресов на шине памяти
- **виртуальное АП (ВАП)** –
АП программы (адресов, содержащихся в командах), которое фрагментарно отображается на ФАП блоками по 4Кб (страницами)
- защита, независимость АП, конкуренция задач за ФАП

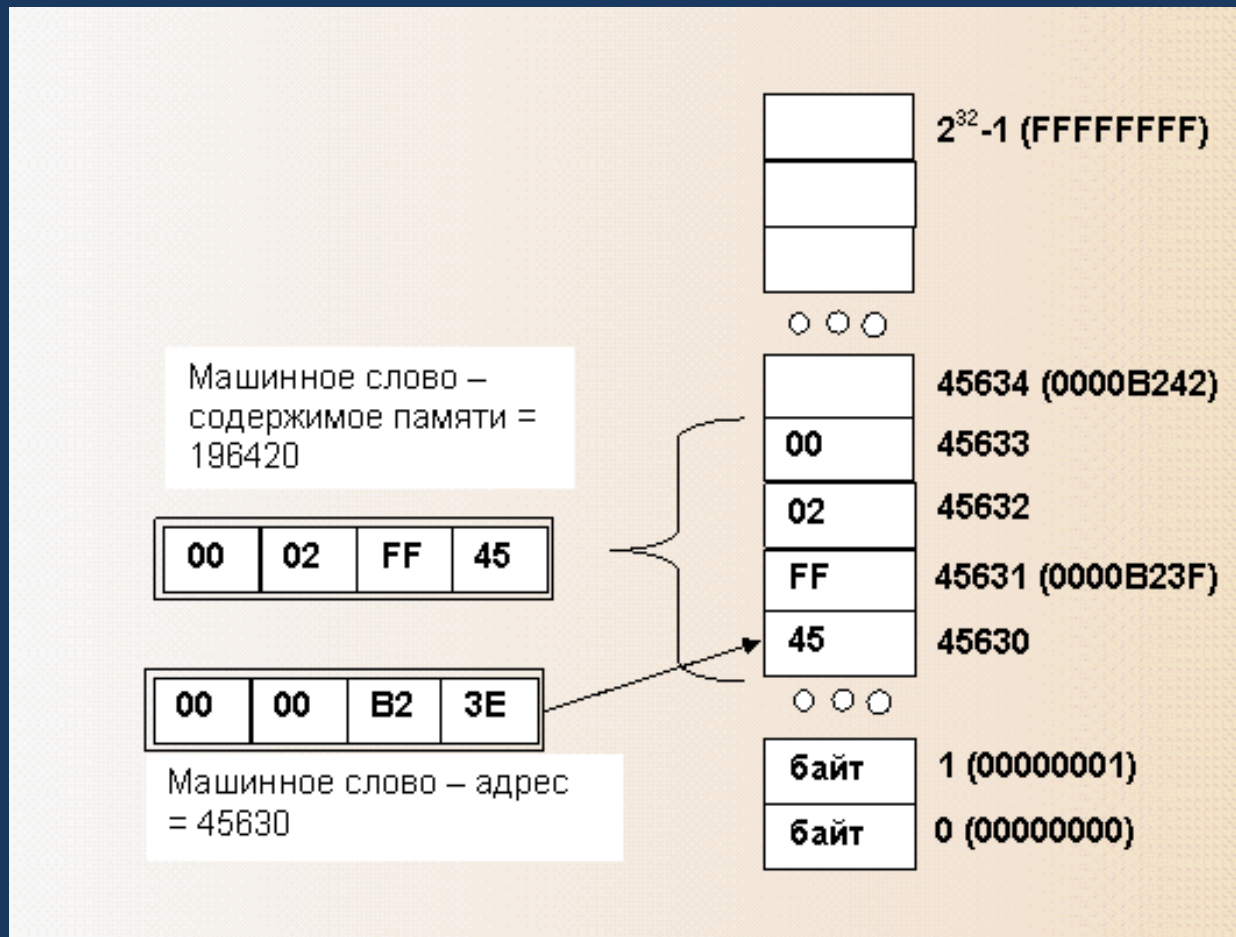




Факты из компьютерной архитектуры

Прямо – адресуемая память

- **адрес** – двоичный (16-ричный) номер байта
- адресное пространство (АП) – непрерывный диапазон адресов

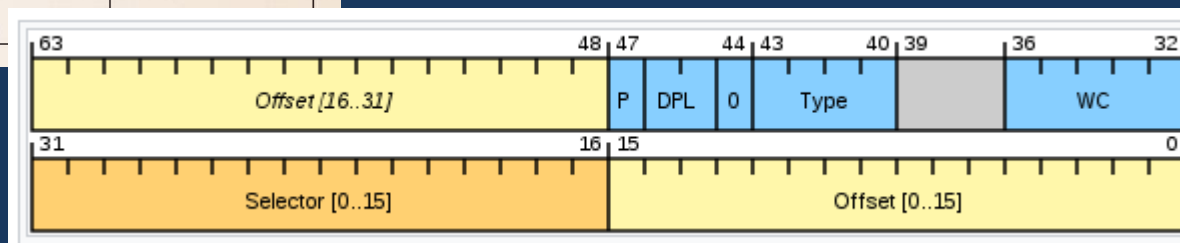
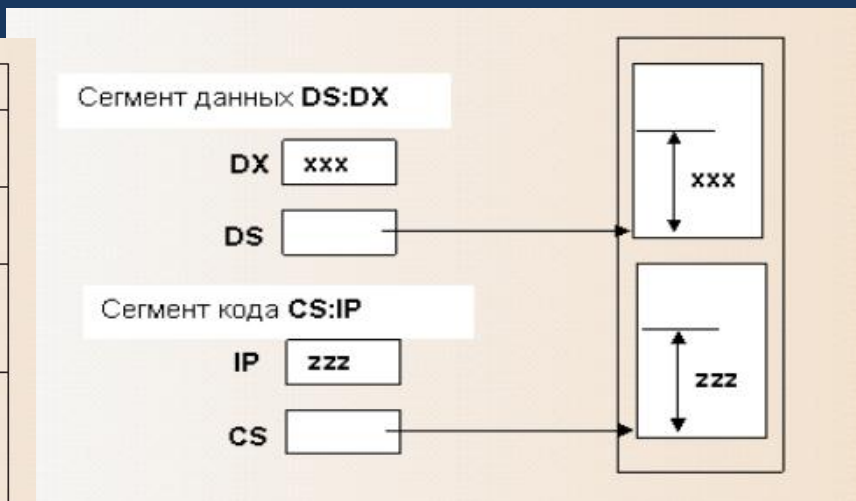




Факты из компьютерной архитектуры

Сегмент - непрерывная область памяти, хранящая данные одного вида (назначения) и имеющая собственную систему относительной адресации и ограничения доступа.

Сегмент	Регистры	Что содержит	Когда создается
Сегмент команд	CS- сегментный, IP- адрес команды	Программный код (операции, операторы)	Трансляция
Сегмент данных	DS – сегментный	Глобальные (статические) данные	Трансляция
Сегмент стека	SS – сегментный, SP- указатель стека	Локальные данные функций, «история» работы программы	При загрузке
Динамическая память	DS – сегментный	Динамические переменные, создаваемые при работе программы	При загрузке, выполнении
Динамически связываемые библиотеки (DLL)	CS- сегментный, IP- адрес команды	Программный код разделяемых библиотек	При загрузке



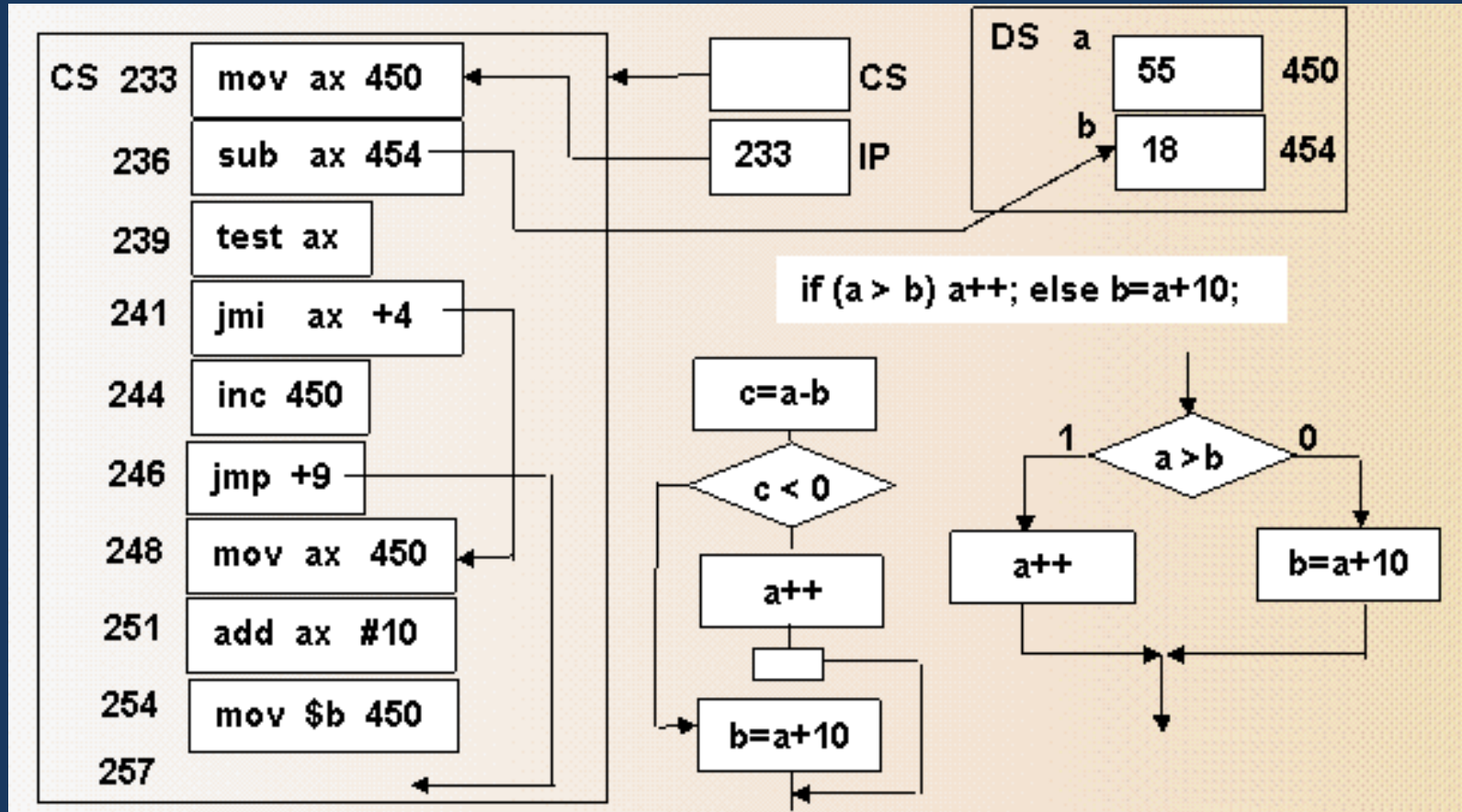
Сегментные регистры содержат **номер сегмента** — **индекс** в таблице описателей сегментов (глобальной - GDT или локальной - LDT)



Факты из компьютерной архитектуры

Внутреннее представление программы

CS – базовый регистр сегмента кода, IP – адрес следующей команды





Факты из компьютерной архитектуры

Система команд процессора

- 1.команды перемещения и обработки (mov,add,sub)
- 2.команды проверки состояний (условий (test)
- 3.команды условного и безусловного переходов

Линеаризованная блок – схема

Способы адресации операндов и команд

- 1.неявный (по умолчанию)
- 2.регистровый
- 3.стековый (операнд в стеке)
- 4.абсолютный – в команде адрес (смещение)
- 5.относительный – смещение от текущей (следующей) команды (IP)
- 6.косвенный (регистровый, абсолютный)
- 7.базовый – относительно начала сегмента, блока, массива



Процессы

Определения, термины

Процесс = память + программа (образ памяти) + ресурсы + потоки

Образ памяти — двоичный файл с содержимым памяти «байт-в-байт»

Процесс — единица планирования в ОС, распределения ресурсов

В ядре — объект-дескриптор процесса (ID процесса) (**объект-процесс**)

ВАП (виртуальное адресное пространство) процесса:

- Уникальное ВАП процесса (00000000...7FFFFFFF)
- Общее ВАП ядра (80000000...FFFFFFFF)

Контекст процесса — структуры данных ядра, развернутые из дескриптора на время выполнения процесса (в т.ч. отображение ВАП на ФАП). Ядро «настроено» на текущий исполняемый процесс

DLL (Dynamic Link Library) - динамически связываемая библиотека, файл-образ памяти с разделяемым программным кодом



Процессы

Режим пользователя (USER) - режим ядра (KERNEL)

Режим пользователя (USER) — «не видит» ядро, **режим ядра (KERNEL)** - «видит» всё — настройка ВАП

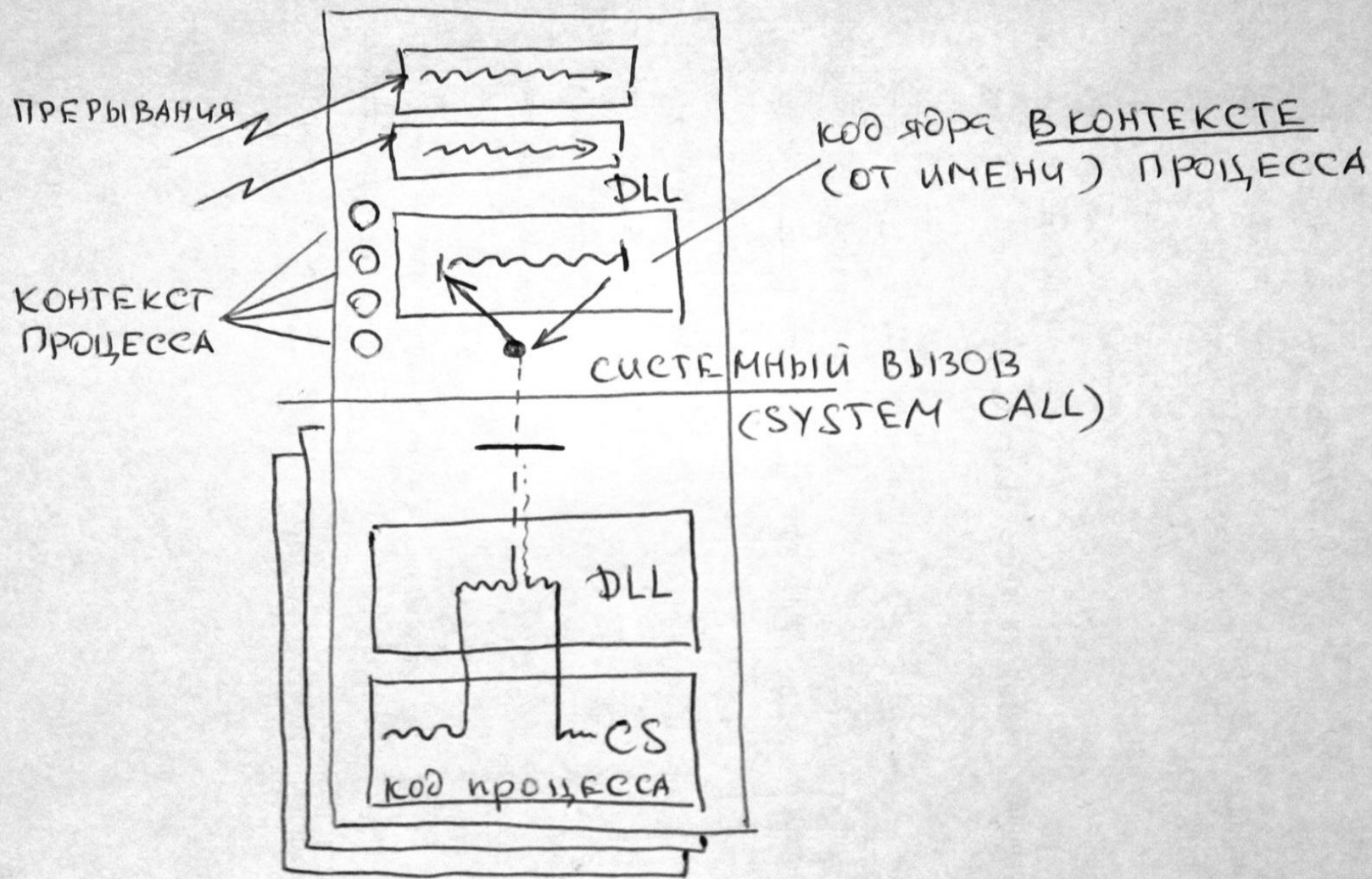
Режим пользователя/ядра (1/0) — состояние ЦП (разряд в слове состояния, разряд в дескрипторах страниц ВАП)

Защита ядра ОС — процесс пользователя может проникнуть в ядро только через системный вызов

Системный вызов (System Call) — программное прерывание, переход в фиксированную точку ядра, выполнение кода ядра в контексте («от имени») процесса, переключение с режима пользователя в режим ядра



Процессы

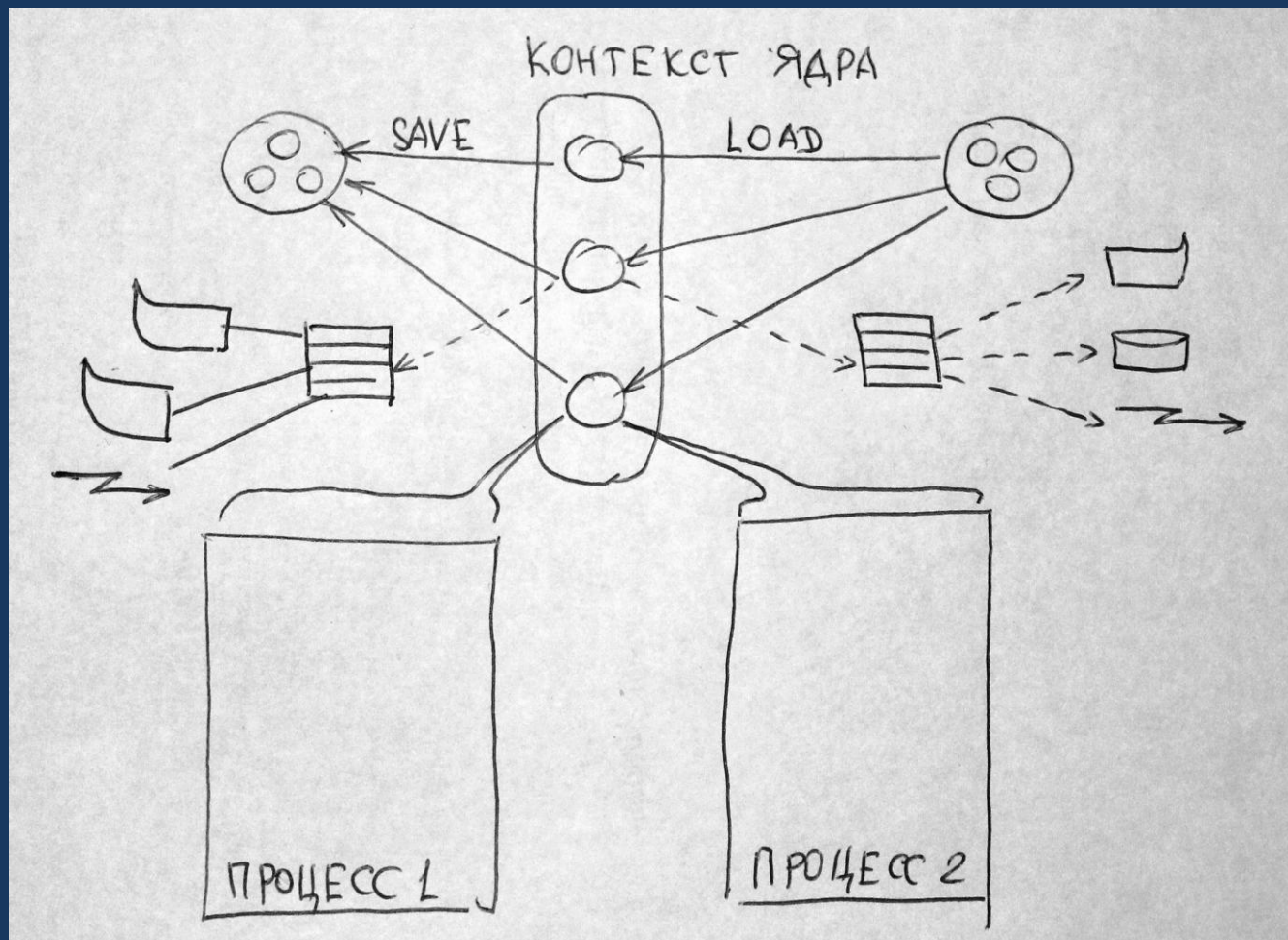




Управление процессом (диспетчеризация)

Переключение процессов — сохранение/восстановление контексте ядра контекста текущего процесса (отображение ВАП, таблицы ресурсов...)

Создание процесса, Уничтожение процесса

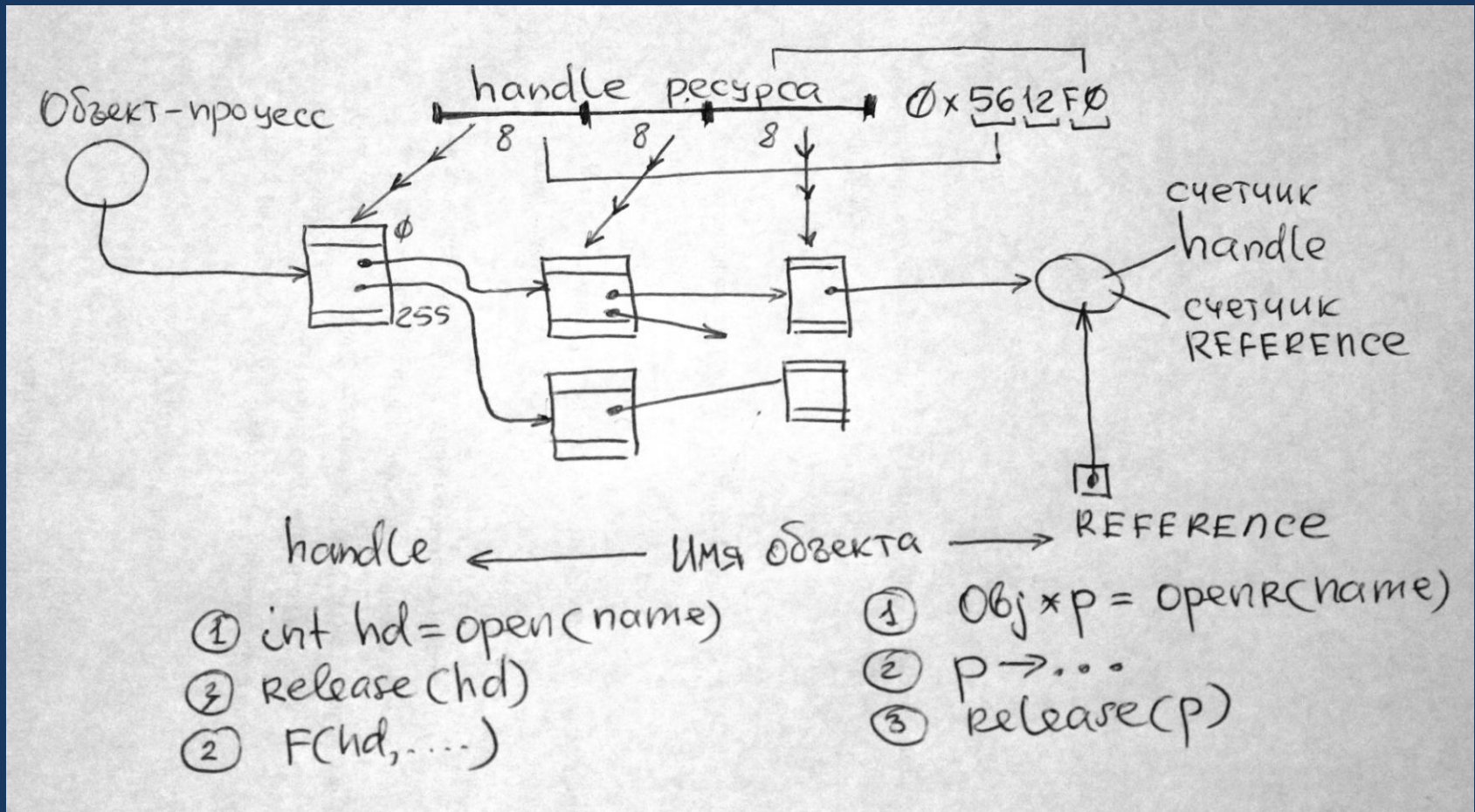




Процесс — владелец ресурсов

Контекст процесса — **таблица ресурсов** - трехуровневый массив указателей (256), 2^{24} — адресуемых объектов ядра (ресурсов)

Handle — целочисленный идентификатор объекта, доступ по handle и по прямой ссылке (reference), контроль ссылок и handle





Управление процессом

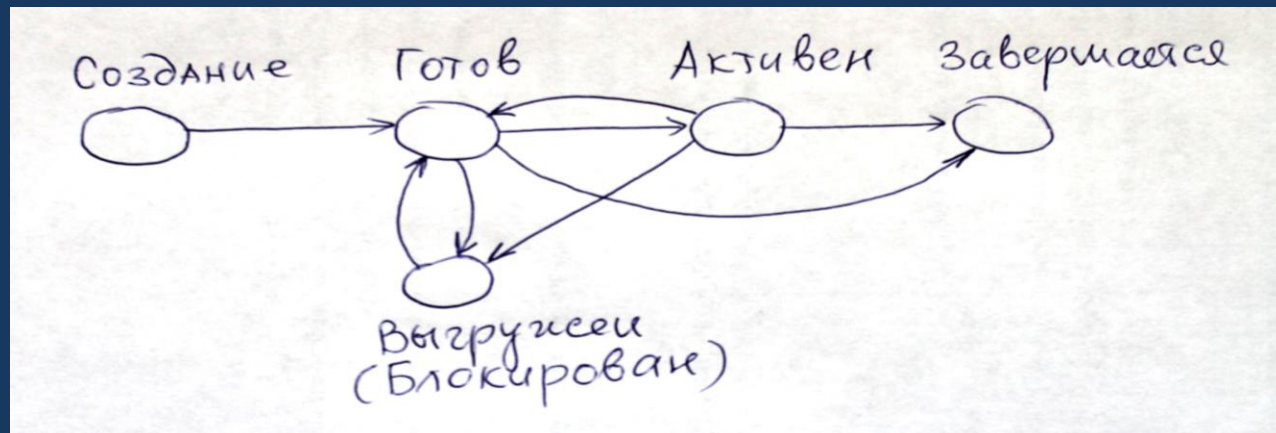
Состояние процесса — множество состояний процесса по отношению к его исполнению: активен, готов, выгружен, завершен - диаграмма состояний)

Диспетчеризация (тактическое управление) — переключение готовых процессов

Планирование (стратегическое управление) — определение порядка и времени развертывания (исполнения) процессов в ядре — режим работы ОС

СД ядра: список объектов-процессов

Блокировка потока средствами синхронизации: ожидание события, критическая секция, «засыпание» на таймере, загрузка страницы ВАП, приостановка (suspend/resume)





Потоки (Threads)

Поток команд — последовательность исполняемых команд в процессе

Поток - «функция, исполняемая параллельно main»

Архитектура ЦП — архитектура потока команд (данные вторичны по отношению к командам)

Поток — единица внутреннего параллелизма в программе

Поток = «история», локальные данные (стек потока) + состояние потока (состояние процессора)

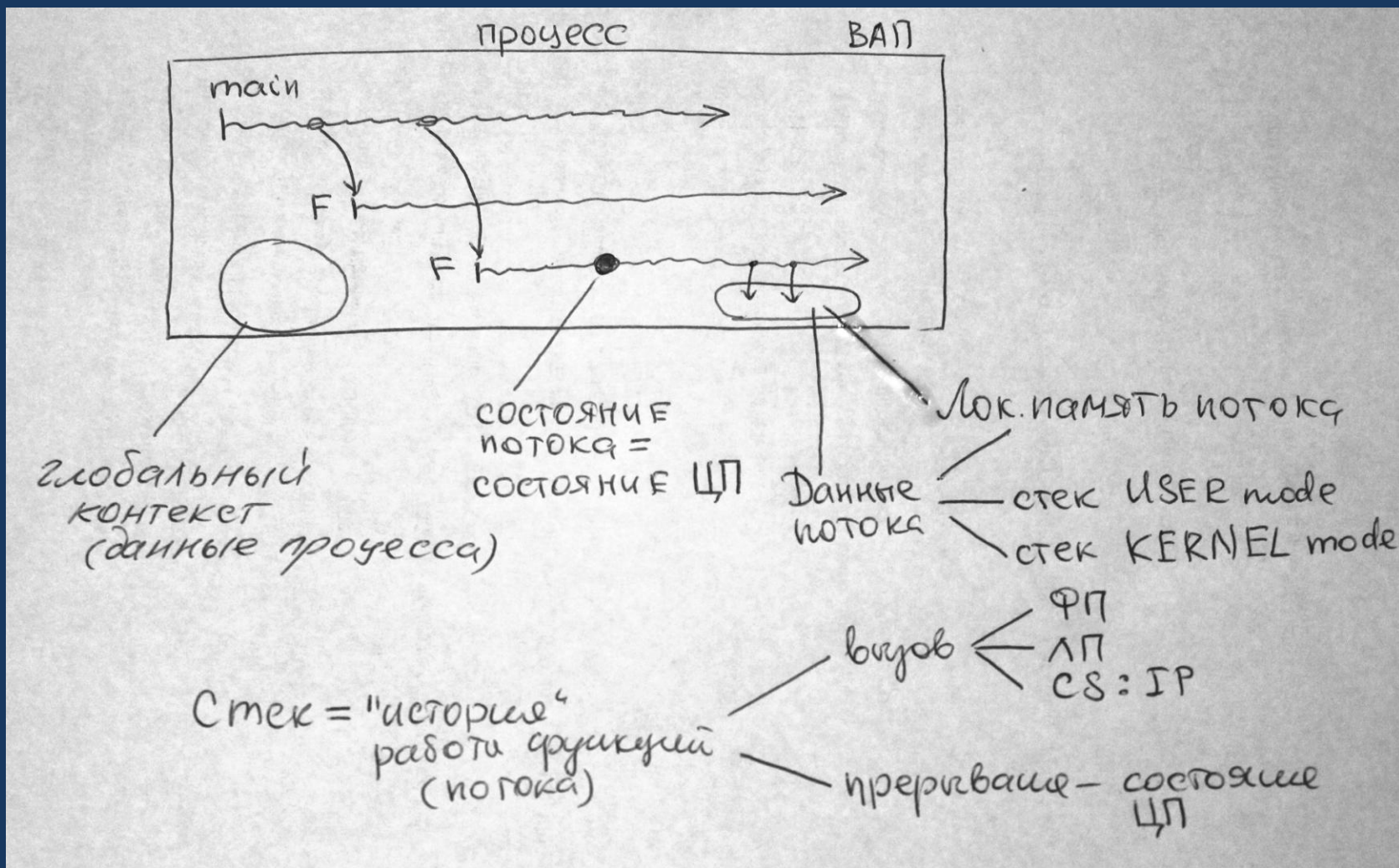
Компоненты потока:

- состояние потока (состояние ЦП в момент прерывания)
- стек режима пользователя, стек режима ядра
- локальная память потока
- объект-поток в СД ядра (дескриптор потока)

Содержимое стека: вызов функции (формальные параметры, CS:IP — точка возврата, локальные переменные), состояние ЦП в точке прерывания



Потоки



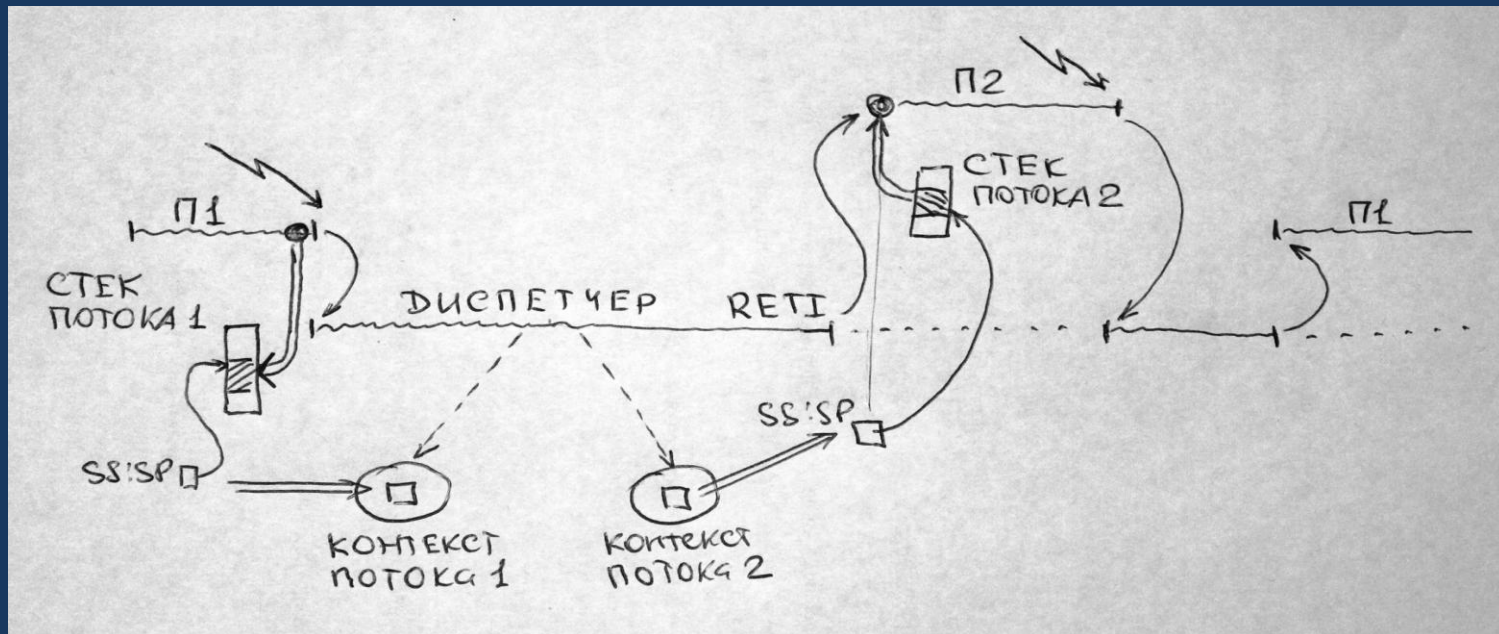


Управление потоком (диспетчеризация)

Состояние потока — состояние процессора в точке прерывания (программно-доступные регистры + регистры состояния)

Переключение потоков: сохранение состояния потока 1 (прерывание), переключение стеков процессора, восстановление состояние потока 2 (выход из прерывания)

Технологический аспект: состояние потока может быть сохранено по прерыванию (асинхронное (часы) или программное — системный вызов, ловушка (trap), либо в точке входа в функцию (эмуляция)





Программирование потоков (C++)

```
#include <stdio.h>
#include <string.h>
#include <winsock2.h>
#include <windows.h>
#include <sys/types.h>
```

```
HANDLE hh = CreateThread(NULL, 0, (LPTHREAD_START_ROUTINE)receiver, NULL, 0, NULL);
if (hh == NULL){
    perror("Не запускается поток приема");
    return 0;
}
return 1;
}
```

```
/** Поток приема */
void *receiver(void *arg){
    while (finish==0){
        Int = recvfrom(sockIn, buf, 512, 0, NULL, NULL);
        finish = (*(int*)buf == OffLine); // Команда ShutDown
        received=1;
    }
    return NULL;
}
```

```
typedef DWORD (__stdcall *LPTHREAD_START_ROUTINE) (
    [in] LPVOID lpThreadParameter
);
```

Параметр запуска потока — функция, исполняемая как поток. Тип данных — указатель на функцию **void (*pf)(...)**



Потоки в Java

Особенности потоков в Java:

- используются реальные потоки ОС и средства синхронизации ОС
- вписаны в парадигму ООП: поток — не функция, а метод `run` класса с присоединенным интерфейсом **Runnable**
- поток состоит из двух компонент — компонента исполняемого кода потока и управляющий объект (дескриптор) — класс **Thread**
- программный код потока — это метод **run** в объекте с присоединенным интерфейсом **Runnable**. **Runnable** — обозначением исполняемого кода (таймер, отложенное исполнение)
- связывание объекта **Thread** с управляющим кодом в методе **run** — наследование или передача ссылки на объект с присоединенным интерфейсом **Runnable**



Потоки в Java

Прямое управление потоком производится по ссылке на объект **Thread** через методы:

- **start** – запуск потока на исполнение
- **interrupt** – деблокирование (пробуждение) потока, если он «заснул» на методе **Thread.sleep**
- не рекомендуемые к использованию методы прямого управления потоком: **suspend** – приостановить, **resume** – возобновить, **stop** – остановить.
- статический метод **Thread.sleep** приводит к блокированию (засыпанию) потока, от имени (в коде) которого исполняется этот метод. Пробуждение потока происходит по истечении интервала времени, либо по выполнению метода **interrupt** над его управляющим элементом. При этом в потоке происходит исключение типа **InterruptedException**
- статический метод **Thread.yield** принудительно переключает процессор с текущего потока на какой-либо другой

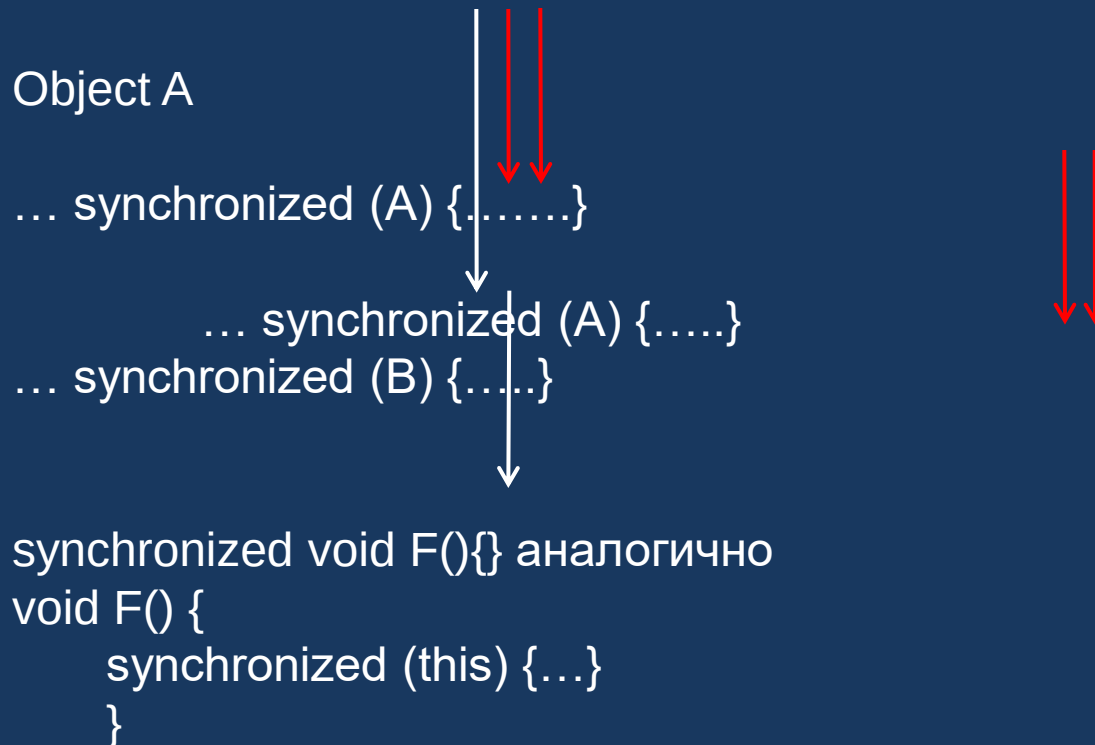
Паттерн двухфазное завершение: Поток нельзя «убить», устанавливается переменная, указывающая на необходимость завершения. Поток, прочитав переменную, завершается с уведомлением основного потока



Потоки в Java

Синхронизация потоков – ограничение на порядок исполнения кода потоков, при разделении ими ресурсов или наличии причинно-следственных связей

1. **Разделение ресурсов**, критическая секция: **synchronized (Object) {...}**
монопольное использование блока одним из потоков (очередь на входе),
Object – объект синхронизации – идентификатор разделяемого ресурса





Потоки в Java

2. Причинно-следственная связь — поток блокирован и ожидает события , происходящего в другом потоке:

- wait — блокировка — ожидание, выполняется на синхронизирующем объекте (**Object A = new Object(); ... A.wait();**)
- Пробуждающий поток выполняет **A.notify()** или **A.notifyAll()**
- **wait** и **notify** должны быть синхронизированы **synchronized (A){...}**
- Поток нельзя «усыпить» принудительно, он может сделать это только сам.
Пример кнопки «Остановить/возобновить»:
 1. Возобновить — поток выполняет **notifyAll()**
 2. Приостановить — поток взводит логическую переменную. Потоки, увидев ее, выполняют **wait**



Потоки в Java

Пример: выполнение продолжительной операции в отдельном потоке (вне потока GUI), пошаговое выполнение, приостановка и прерывание

- Главное окно запускает в потоке операцию, передает объект связи `ViewAdapter.viewCommon`, который одновременно является объектом синхронизации и содержит переменные управления исполнением
- Нажатие кнопок приводит к изменению переменных в `ViewAdapter` или к пробуждению потока, «уснувшего» на объекте
- Исполняемая операция периодически вызывает переопределенные методы в `ViewAdapter`, если установлены соответствующие значения переменных, то поток «засыпает» на этом объекте

```
new Thread(  
    ()->{operate.sliceTo(fname,gen,mode,Optimize.isSelected(),viewCommon);  
        finishOperation();  
        notify.notify(Values.important, mes: "Время выполнения: "+viewCommon.timeInMs()/1000+" сек.\n");  
    }).start();  
}
```

```
/** пустой адаптер для связи с View */  
public class ViewAdapter {  
    /** Панель для рисования */  
    public final JPanel FLD;  
    /** размер панели */  
    public final int sz;  
    /** масштаб рисования */  
    public volatile double mas;  
    /** признак - процесс выполняется */  
    public volatile boolean running;  
    /** признак - процесс завершен */  
    public volatile boolean finish;  
    /** признак - процесс приостановлен */  
    public volatile boolean pause;  
    /** задержка (мс) после отрисовки линии */  
    public int lineDelay = 0;  
    /** задержка (мс) после отрисовки слоя */  
    public int layerDelay = 10;  
    /** системное время старта */  
    public long startTime=0;
```



Потоки в Java

```
private void STOPActionPerformed(java.awt.  
    if (!viewCommon.running)  
        return;  
    if (viewCommon.pause){  
        STOP.setText("остановить");  
        synchronized (viewCommon)  
            { viewCommon.notifyAll();}  
        }  
    else{  
        STOP.setText("продолжить");  
        }  
    viewCommon.pause=!viewCommon.pause;  
} //GEN-LAST:event_STOPActionPerformed
```

```
notify = new ViewNotifier(LOG){  
    @Override  
    public void notify(final int level0, String mes) {  
        super.notify(level0, mes);  
        java.awt.EventQueue.invokeLater(  
            ()->{  
                if (level0>=Values.error){  
                    viewCommon.finish=false;  
                    viewCommon.running=false;  
                    viewCommon.pause=false;  
                    STOP.setText("...");  
                }  
                if (level0 >Values.warning || stopOnWar  
                    if (viewCommon.running) {  
                        viewCommon.pause = true;  
                        STOP.setText("продолжить");  
                    }  
            }  
        });  
    }  
};
```

```
viewCommon = new ViewAdapter(FLD,sz,mass) {  
    @Override  
    public void onStepLine() {  
        try {  
            if (BYSTEP.isSelected()){  
                pause=true;  
                STOP.setText("продолжить");  
            }  
            if (pause)  
                synchronized (viewCommon)  
                    { viewCommon.wait();}  
            if (lineDelay!=0)  
                Thread.sleep(lineDelay);  
        } catch (InterruptedException ex) {}  
    }  
}
```

```
@Override  
public boolean isFinish() { return finish; }  
@Override  
public void onStepLayer() {  
    try {  
        if (BYSTEP.isSelected()){  
            pause=true;  
            STOP.setText("продолжить");  
        }  
        if (pause)  
            synchronized (viewCommon)  
                { viewCommon.wait();}  
        if (layerDelay!=0)  
            Thread.sleep(layerDelay);  
    } catch (InterruptedException ex) {}  
}
```

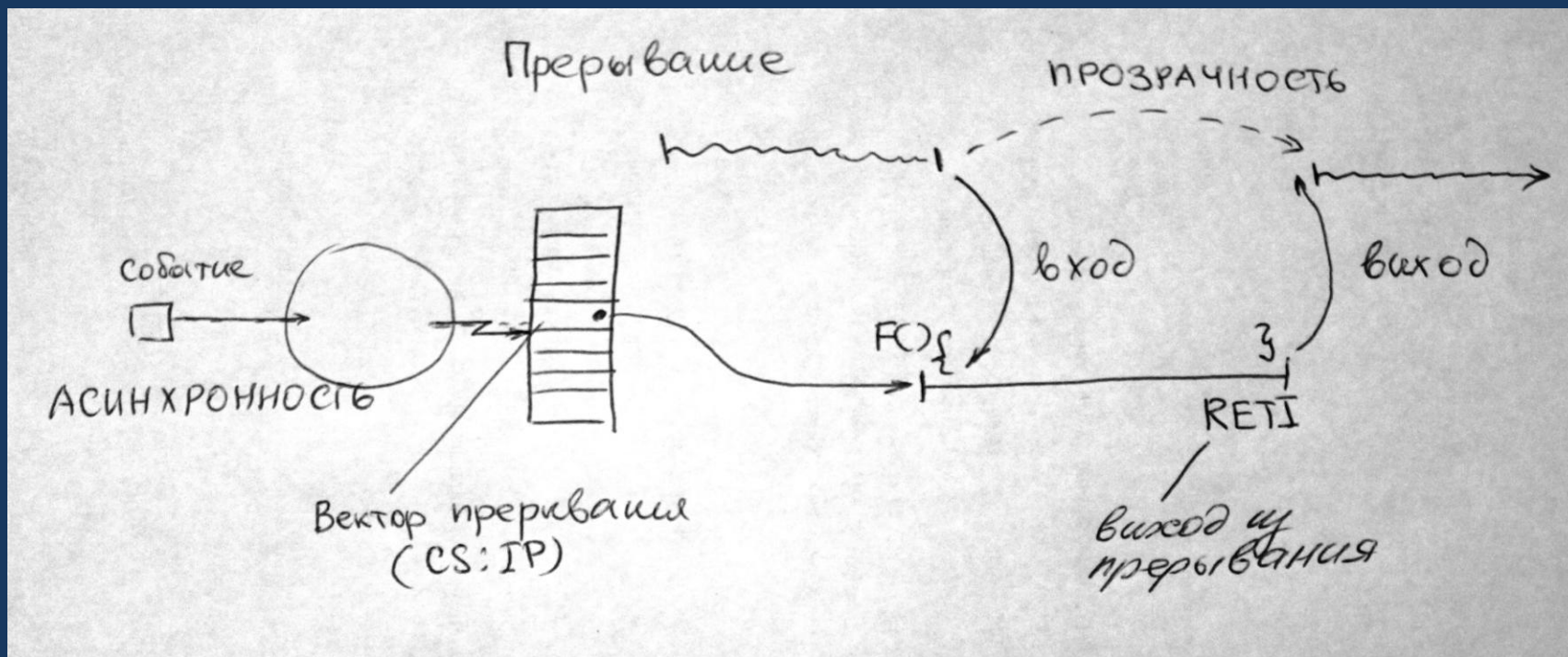


Прерывание (Interrupt)

Прерывание — асинхронная прозрачная процедура, вызываемая внешним событием

Прозрачность — прерываемый поток «не видит» прерывания. Вход и выход в прерывание — **сохранение и восстановление состояния процессора**

Асинхронность — событие в произвольный момент времени относительно исполнения прерываемого потока





Прерывания

Сохранение состояния процессора:

- Минимально-аппаратно — CS:IP + слово состояния процессора (режим), остальное программно
- Максимально-аппаратно: **TSS - TaskStateSegment**

TSS — сегмент, содержащий состояние процессора для некоторой задачи. LINK — сегмент TSS предыдущей (прерванной) задачи.

Функция обработки прерывания (ISR) — функция, завершающаяся командой (кодом) выхода из прерывания

Связь источника прерываний с функцией обработки - **таблица векторов прерываний** (IVT — 4B, реальный режим 8086), **дескрипторов прерываний** (IDT — Interrupt Descriptor Table, 8B, защищенный режим)

Номер вектора прерывания — идентификатор источника прерываний = Индекс в IDT (IVT)

x86 Structure

offset	31-16	15-0
0x00	reserved	LINK
0x04	ESP0	
0x08	reserved	SS0
0x0C	ESP1	
0x10	reserved	SS1
0x14	ESP2	
0x18	reserved	SS2
0x1C	CR3	
0x20	EIP	
0x24	EFLAGS	
0x28	EAX	
0x2C	ECX	
0x30	EDX	
0x34	EBX	
0x38	ESP	
0x3C	EBP	
0x40	ESI	
0x44	EDI	
0x48	reserved	ES
0x4C	reserved	CS
0x50	reserved	SS
0x54	reserved	DS
0x58	reserved	FS
0x5C	reserved	GS
0x60	reserved	LDTR
0x64	IOPB offset	reserved



Прерывания в IA-32

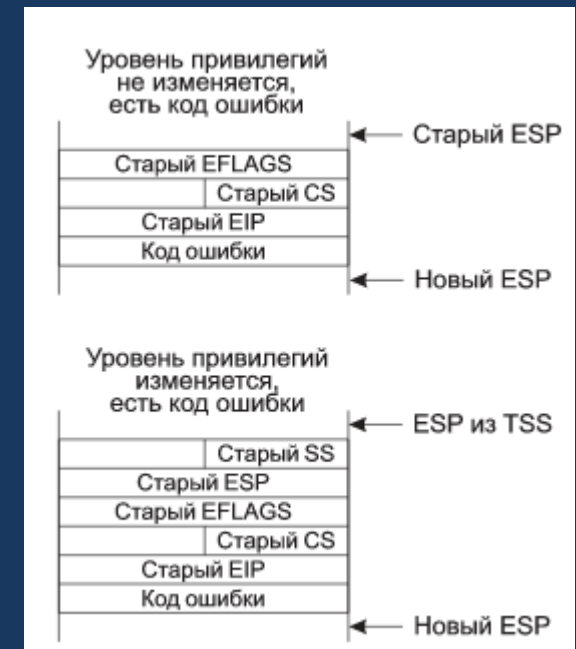
Механизм используется для переключения потоков (задач), обработки прерываний

IDTR — физический адрес IDT

IDT — содержит 256 дескрипторов

Дескриптор — **шлюз задачи** (сегмент TSS) — переключение на задачу с состоянием в адресуемом TSS, сохранением текущего состояния с момент прерывания в TSS старой задачи

Дескриптор — **шлюз прерывания / шлюз ловушки** — переключение стека при изменении привилегий (сохранение в новом стеке SS:SP), CS:IP + FLAGS, код ошибки. Смены контекста задачи и ВАП не происходит





Виды прерываний

Программное прерывание — команда $INTm$ вызывает прерывание по соответствующему ВП, системный вызов, $m=32-255$

Ловушка (trap) — аналог программного прерывания для отладки, трассировки

Внешнее маскируемое прерывание — асинхронное прерывание от ВУ, таймера

Внешнее немаскируемое прерывание — NMI, $m=2$

Программные ошибки и исключения — $m=0-31$ (деление на 0, страничное прерывание, защита памяти)



Аппаратные (внешние) прерывания

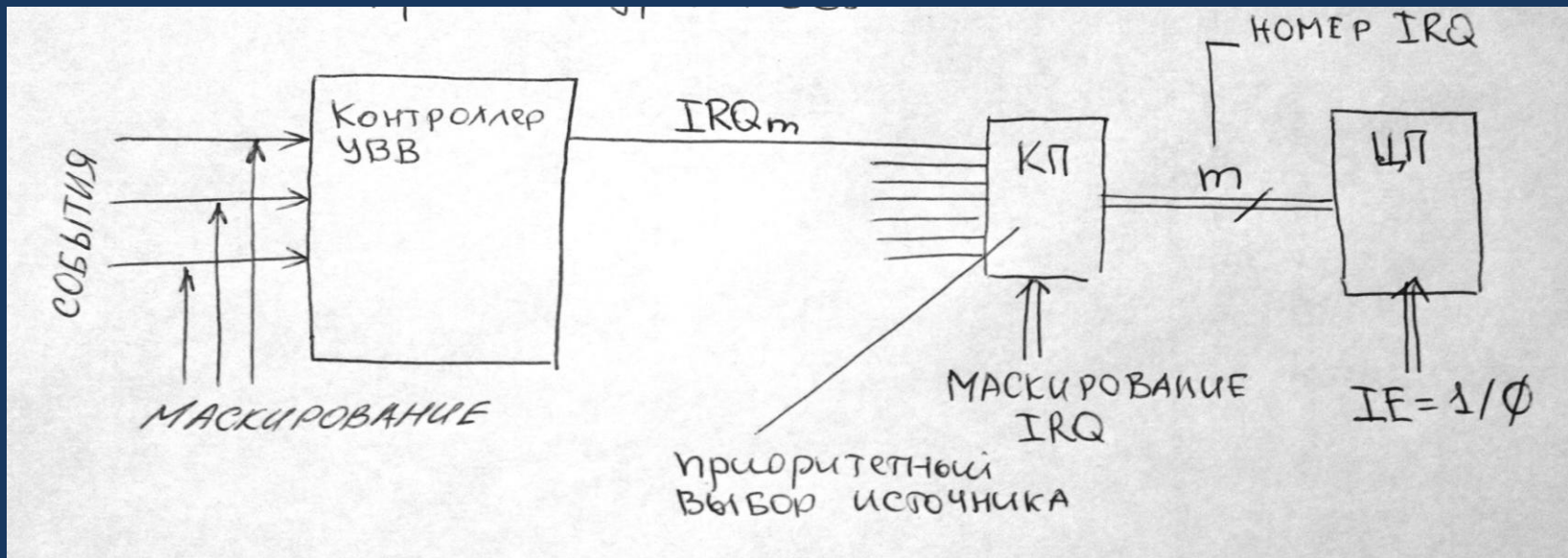
Источники прерываний (флаги ошибок, готовности)

Маскирование прерываний в контроллере УВВ (драйвер)

Единый запрос на прерывание от устройства (**IRQ m — Interrupt ReQuest**), m — номер запроса на прерывание = номер ВП

Контроллер прерываний (APIC, 8259) маскирует источники, определяется источник с максимальным приоритетом (*приоритет выбора*), формирует INTR — запрос прерывания

Обработка прерывания при IE=1 — передача контроллером m — номера IRQ



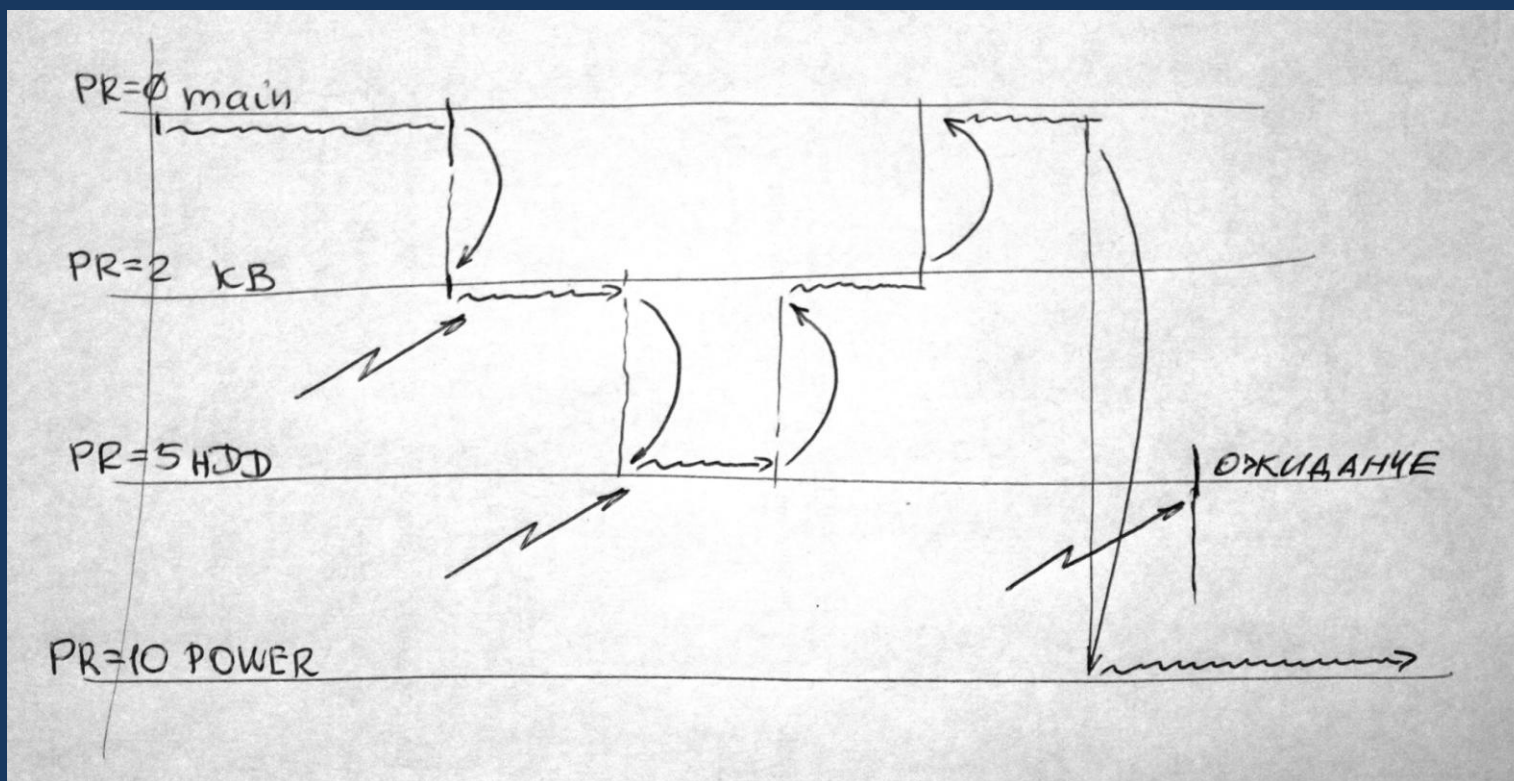


Вложенные прерывания

«Прерывание в прерывании»

Требуется наличие приоритета процессора и приоритета запроса

В IA-32 реализуется программно путем перепрограммирования масок IRQ в КП в соответствии с программным уровнем приоритета





Уровни прерываний в ядре Windows

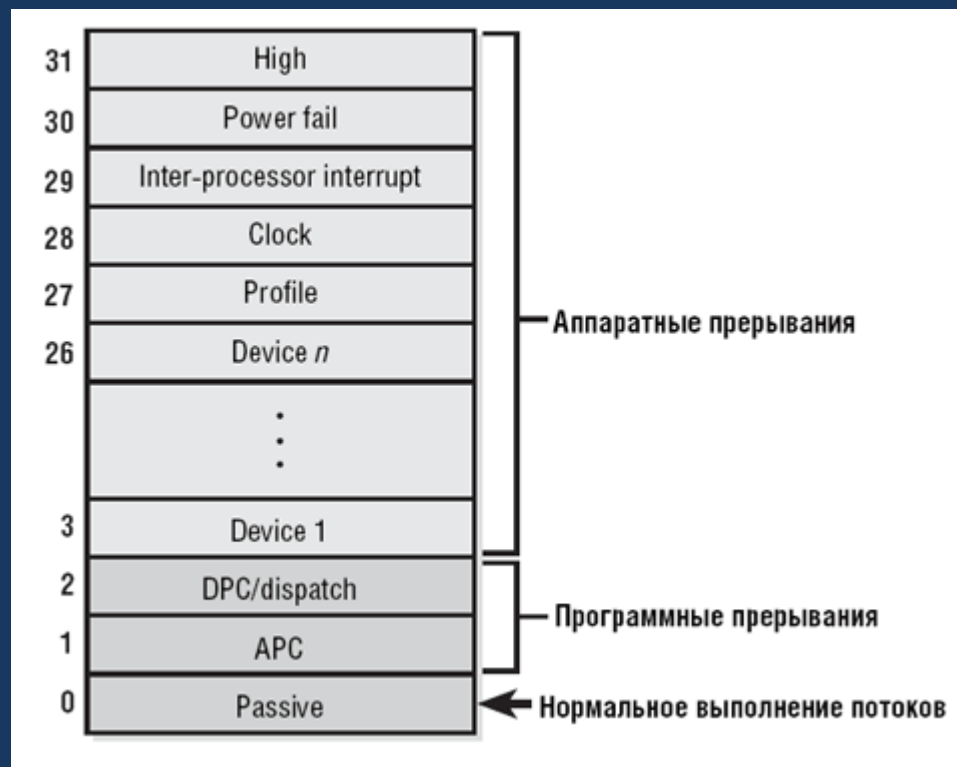
Программный уровень приоритета исполняемого кода (диспетчер прерываний).

IRQL — Interrupt ReQuest Level

Используется для синхронизации компонент ядра, процессов (потоков) и прерываний ВУ в ядре

Программные уровни:

- Passive — поток (процесс)
- ACP — асинхронный вызов процедуры в контексте потока
- DPC — отложенный вызов процедуры — синхронизированная последовательность вызовов функций, разделение СД ядра





DPC — Deffered Procedure Call

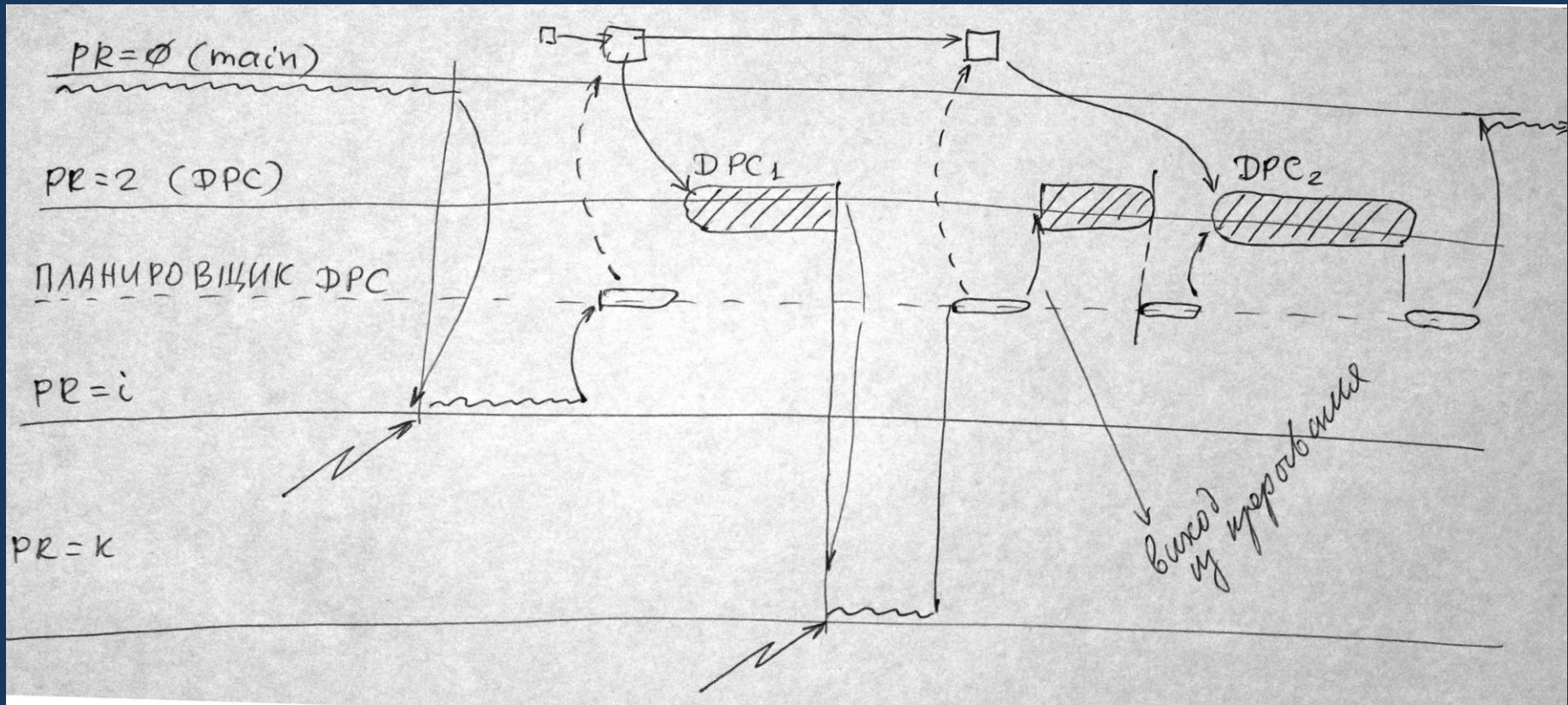
«Отложенное прерывание» — планировщик с очередью исполнения запросов на низком приоритете:

- Приоритет ниже всех аппаратных прерываний, но выше приоритета исполнения процессов
- Запрос DPC из ISR ставится в очередь
- DPC выбираются из очереди и выполняются перед выходом из прерывания с понижением приоритета ядра (при разрешенных прерываниях)
- *Используются для синхронизации к СД ядра (разделение, критическая секция)*
- Процессы планируют DPC через программное прерывание
- *Паттерн параллелизма - «планировщик» (Scheduler)*



DPC — Deferred Procedure Call

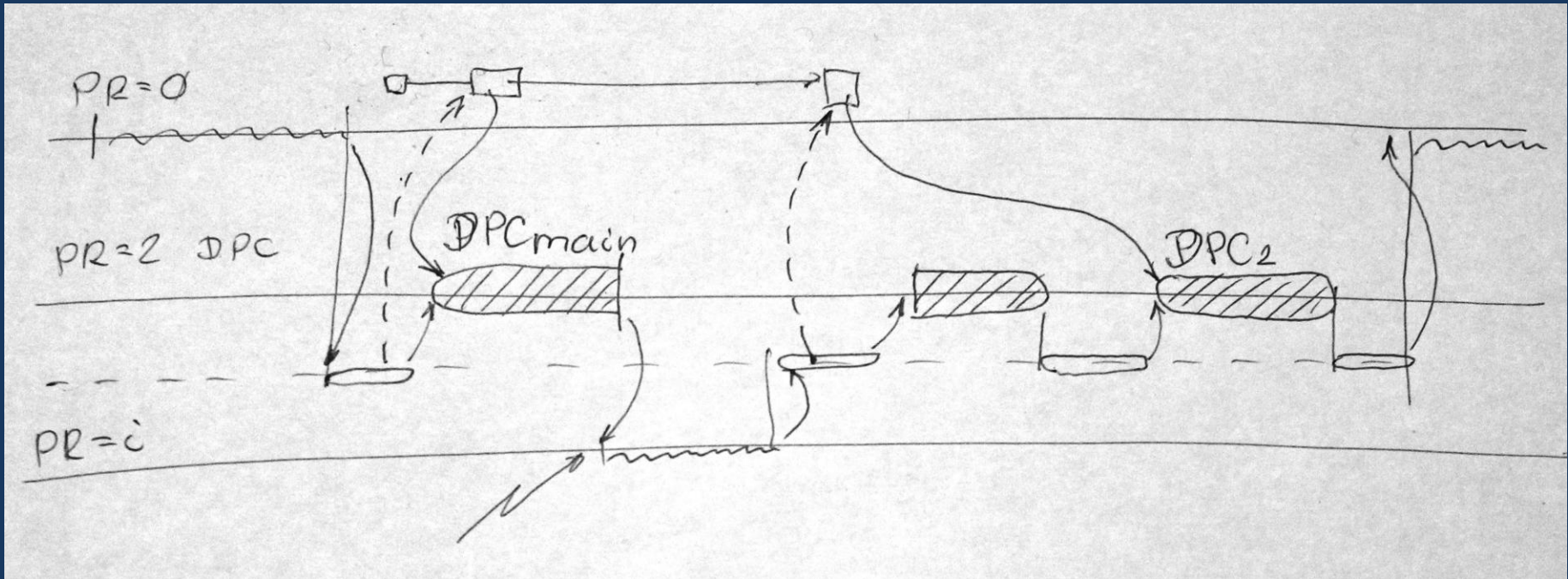
Планирование DPC из прерываний - «отложенное прерывание», работа со СД ядра при завершении обработки прерываний (компонента драйвера)





DPC — Deferred Procedure Call

Планирование DPC из потока (процесса) — синхронизация со СД ядра с системой прерываний





APC — Asynchronous Procedure Call

Асинхронный вызов процедуры — процедура (функция), вызываемая в контексте процесса:

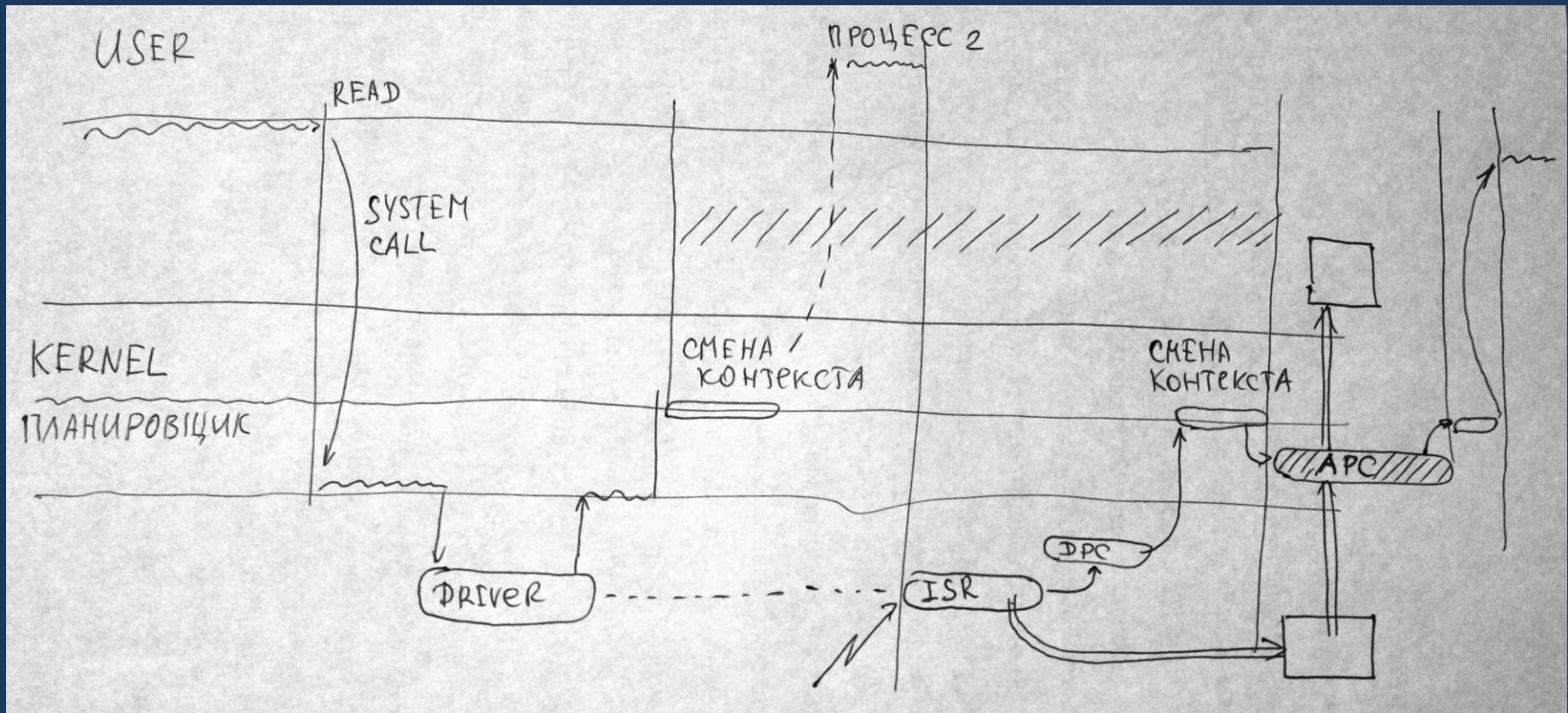
- Компонента кода ядра (ВАП яда) — завершение ввода/вывода, передача данных из буфера кэша ядра в буфер ВАП процесса (чтение данных)
- Компонента кода процесса — асинхронная функция завершения ввода/вывода, **сигнал** (UNIX), аналог прерывания на уровне прикладной программы
- Реализуется диспетчером / планировщиком процессов (поток)



APC — Asynchronous Procedure Call

Асинхронный вызов процедуры — завершение ввода/вывода:

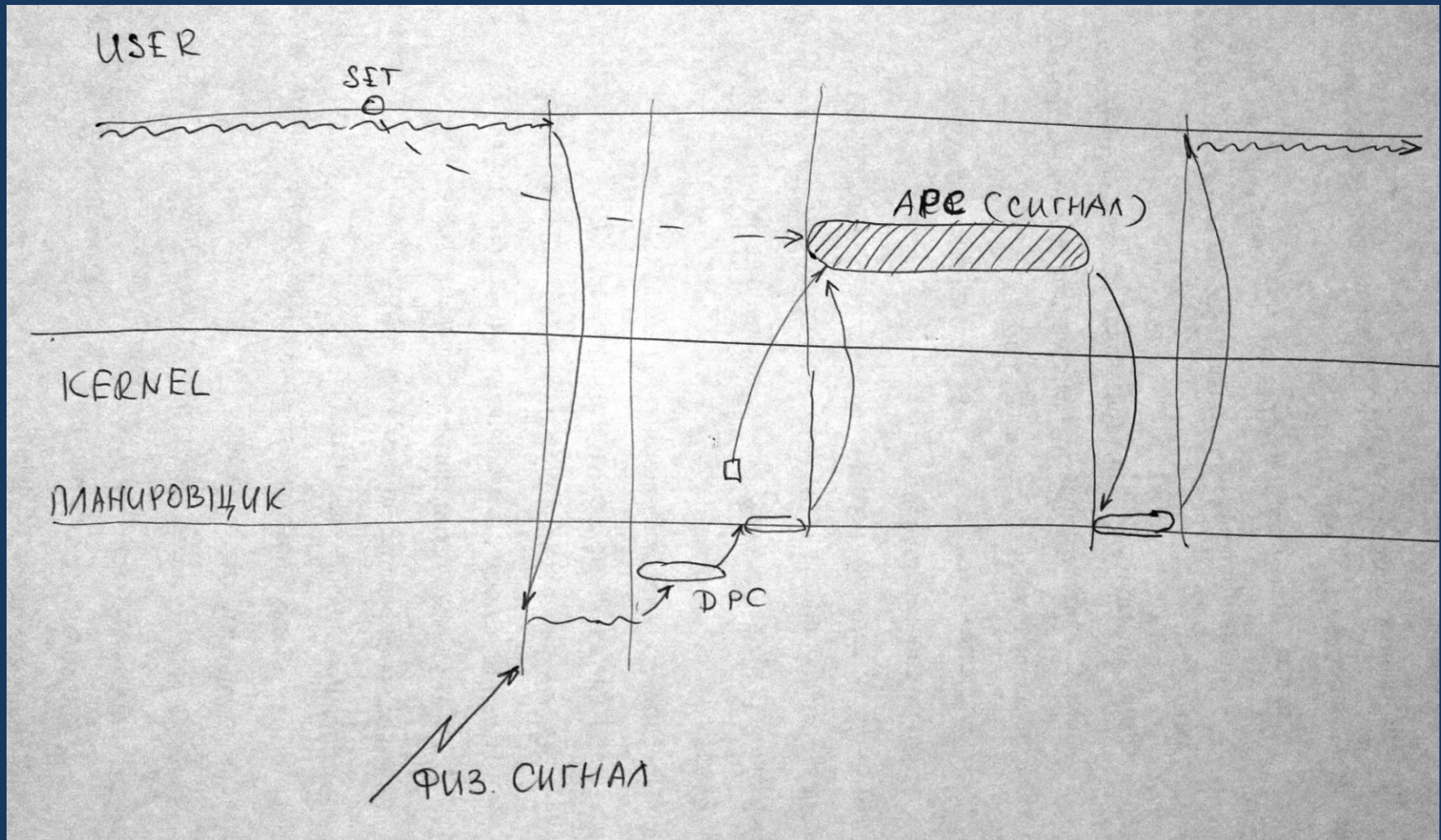
- Драйвер (устройство) читает данные в кэш ядра (контекст процесса может быть недоступен)
- Для копирования данных в буфер программы в ВАП процесса требуется вызывать APC копирования в коде ядра в контексте процесса





APC — Asynchronous Procedure Call

Асинхронный вызов процедуры — асинхронная функция завершения ввода/вывода, **сигнал** (UNIX) по внешнему асинхронному событию





Управление процессами/потоками

Тактическое управление (диспетчеризация) — сохранение/восстановление, переключение, изменение состояния

Стратегическое управление (планирование) — определение последовательности и продолжительности исполнения.

Критерий эффективности планирования — определяется режимом работы ОС

ОС пакетной обработки — эффективность использования ресурсов

ОС разделения времени - «справедливость» распределения ресурсов, интерактивность (время ответа)

ОС реального времени — гарантированное время реакции на внешние события

Планирование в ОС пакетной обработки:

- **Расписание** порядка и времени запуска процессов
- **Пакет** — набор одновременно выполняемых процессов, сбалансированный по использованию процессора, ввода/вывода (очереди) и памяти



Реальное время

Приоритетное планирование — процесс (поток) имеет параметр — приоритет выполнения

Алгоритм планирования: выбирается готовый процесс с максимальным приоритетом

При завершении/блокировании потока — повторное планирование

Вытесняющая диспетчеризация (абсолютные приоритеты) — при появлении процесса с более высоким приоритетом — переключение

Невытесняющая диспетчеризация (относительные приоритеты) — при появлении процесса с более высоким приоритетом — повторное планирование по завершении/блокировании текущего

Обработчики прерываний в ядре ОС (ISR) — подсистема реального времени с абсолютными приоритетами



Разделение времени

Круговая диспетчеризация:

- очередь процессов (поток)
- извлечение из очереди и предоставление кванта
- постановка в конец очереди
- «абсолютная справедливость»

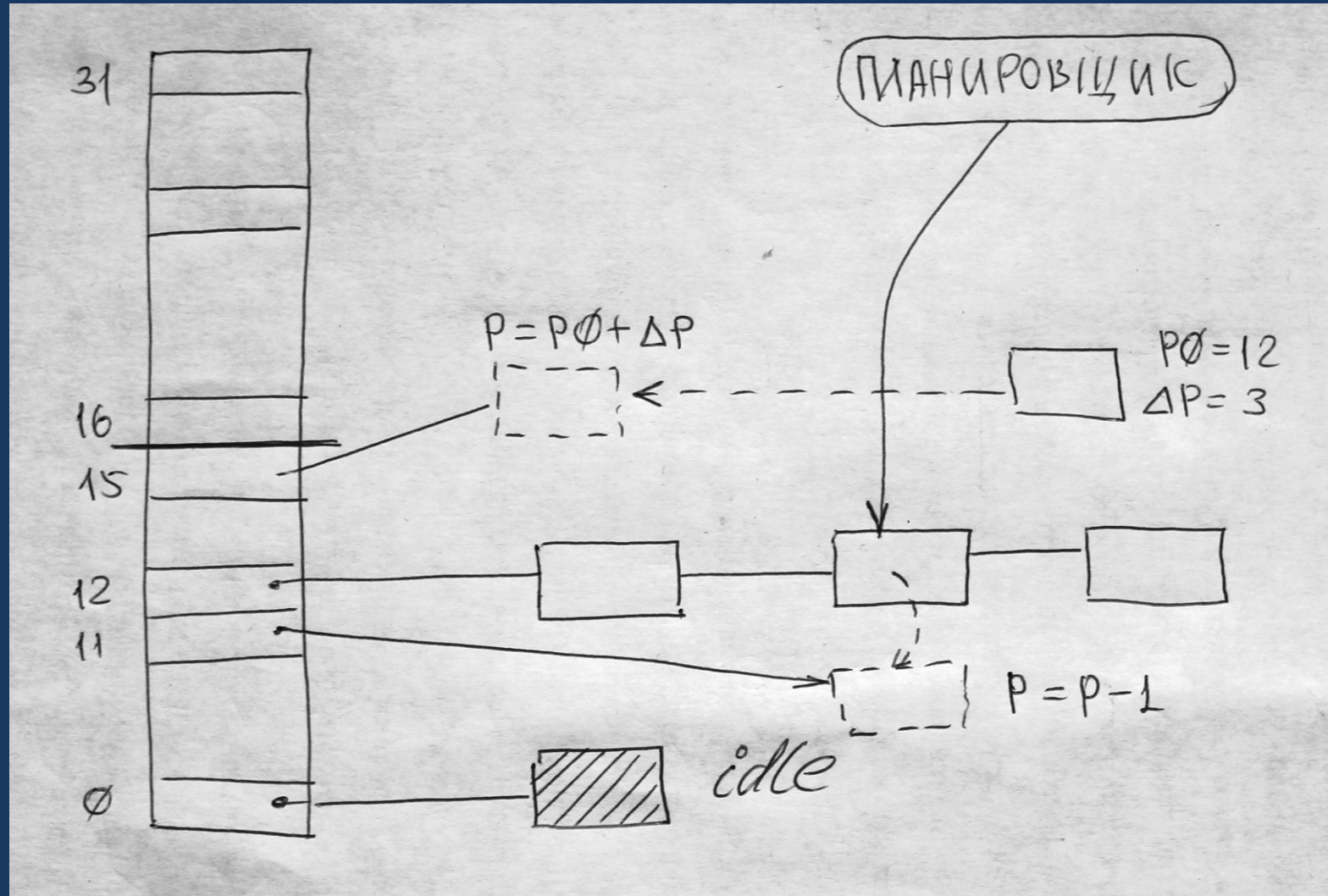
Реальность:

- Система прерываний как компонента реального времени
- Процессы реального времени, в т.ч. интерактивные «игрушки», плееры
- Интерактивные процессы — реакция на некоторые события (GUI), в остальном - «справедливость»
- Фоновые процессы — неинтерактивные, без жестких требований к скорости выполнения и реакции на события
- Idle - «когда системе нечего делать»



Планирование потоков в Windows

Система приоритетных очередей — 32 приоритета, для каждого — собственная очередь потоков





Планирование потоков в Windows

Приоритеты — **базовый, текущий** (текущий динамически повышается)

Алгоритм планирования:

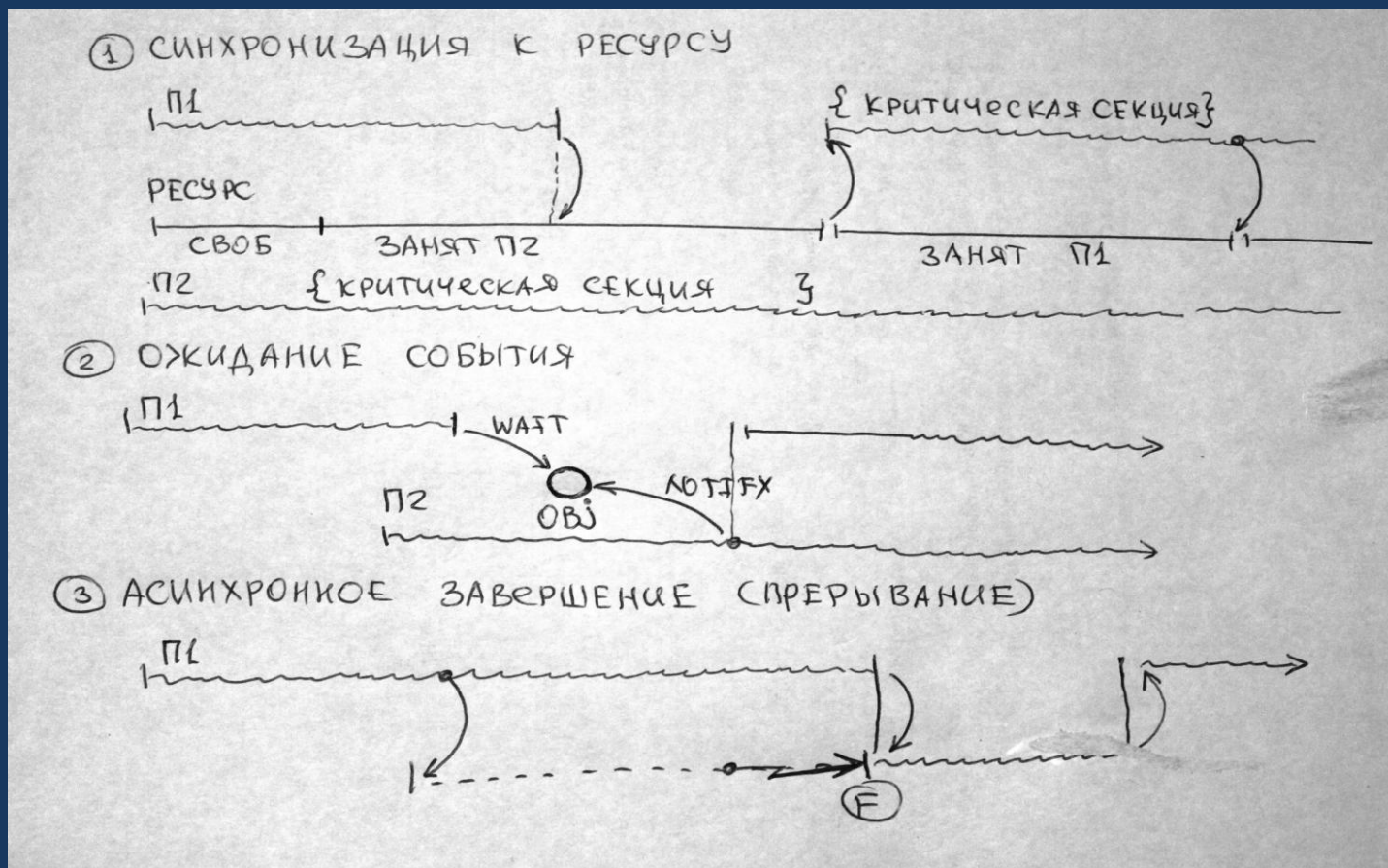
- Выбор непустой очереди с максимальным приоритетом
- Квантование (круговая диспетчеризация) внутри очереди
- Вытеснение при появлении потока с более высоким приоритетом
- Динамическое повышение текущего приоритета повышается относительно базового по событиям:
 - Завершение ожидания В/В (различная величина для разных типов УВВ)
 - Окончание ожидания на событии и семафоре (на 1)
 - Пробуждение GUI-потока
 - Продолжительной готовности без исполнения на низком приоритете (однократно до 15)
- После получения очередного кванта текущий приоритет снижается на 1 (переход в другую очередь)



Синхронизация потоков

Синхронизация и асинхронное исполнение:

- Синхронизация разделения (критическая секция, неделимая операция)
- Причинно-следственная связь (ожидание события)
- Асинхронное завершение (прерывание, обратный вызов - Callback)





Примитивы синхронизации

Пример ошибки синхронизации (покупка билета):

```
int ticket[]={0,0,0,0,0,0};  
if (ticket[k]==0){  
    ticket[k]=userId;  
    nTicket=k;  
}
```

```
if (ticket[k]==0){
```

...переключение потоков — поток 2

```
    if (ticket[k]==0){  
        ticket[k]=userId; // Покупка билета k  
        nTicket=k;  
    }
```

...переключение потоков — поток 1

```
    ticket[k]=userId; // Покупка билета k  
    nTicket=k;  
}
```

Неделимая операция: проверка и установка

```
int busy=0;  
if (busy==0) busy=1;  
  
busy=0;
```

Неделимая операция

```
while(true){  
    if (busy==0)  
        { busy=1; break;   
    }  
}
```

```
busy=0;
```



Windows — ожидание событий на объектах

Аналог без блокировки:

```
void wait(int *obj){ *obj=0; while(*obj==0); }
```

```
void notify(int *obj){ *obj=1; }
```

```
int x=0;
```

```
Поток 1: ... wait(&x);
```

```
Поток 2: ... wait(&x);
```

```
Поток 3: ... notify(&x);
```



Windows — ожидание событий на объектах

Объектная модель ядра: все компоненты — объекты соответствующих классов (процессы, потоки, файлы, семафоры, таймеры, событие, раздел памяти)

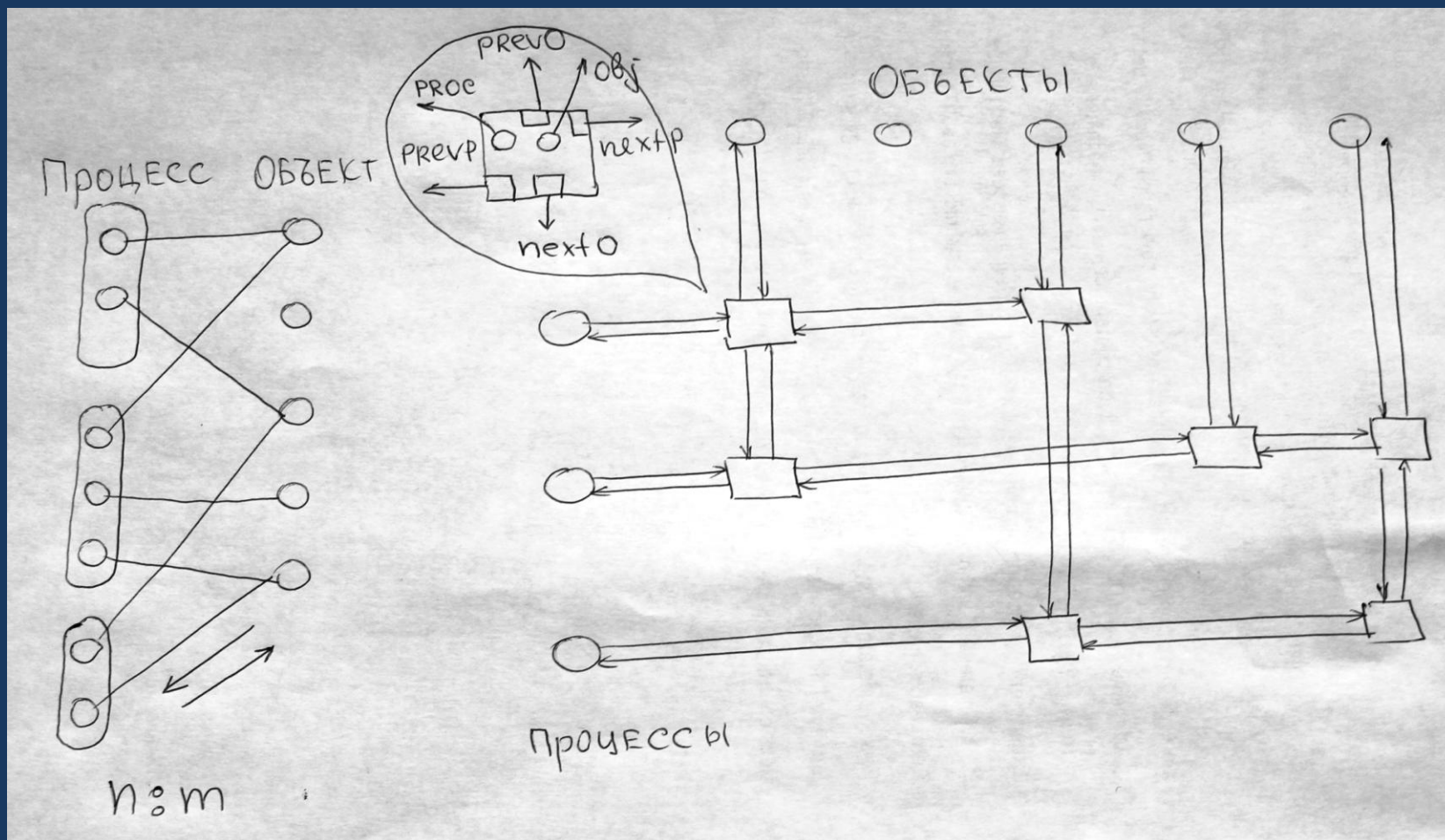
Объект	Событие	notify
Процесс	Завершение всех потоков	все
Поток	Завершение потока	все
Файл	Завершение в/в	все
Событие	Установление события потоком	поток
Семафор	V-операция	поток
Таймер	Истечение интервала	поток (все)



Ожидание событий на объектах

Реализация отношения N:M: структура данных — 2D система списков

Элемент списка (2*2 указателей на соседей + 2 указателя на дескрипторы)



*) Блокируются потоки, но ждут процессы



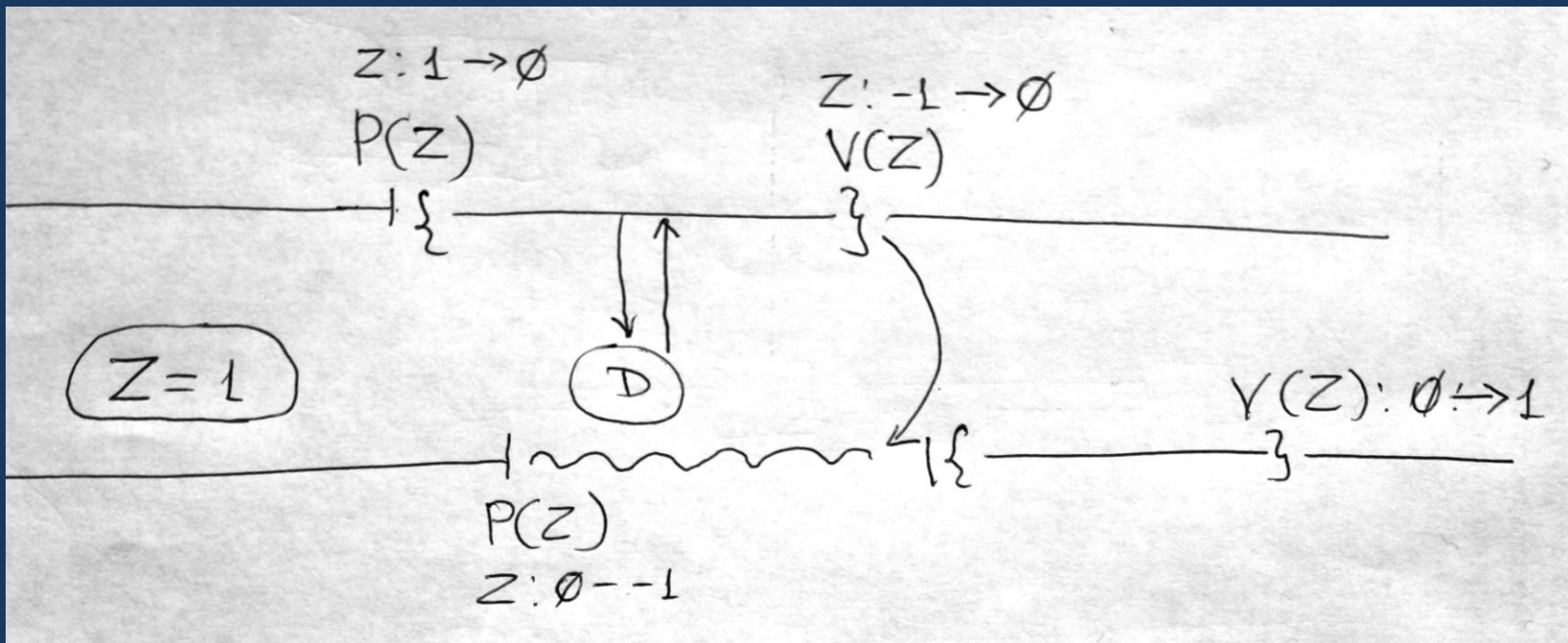
Семафоры

Семафор — объект синхронизации, имеющий N единиц ресурса - $X(N)$

P-операция: $P(X): X = X - 1$ if $(X < 0)$ блокировка потока

V-операция: $V(X): X = X + 1$ if $(X \geq 0)$ разблокирование потоков, ожидающих на семафоре

Единичный семафор — критическая секция

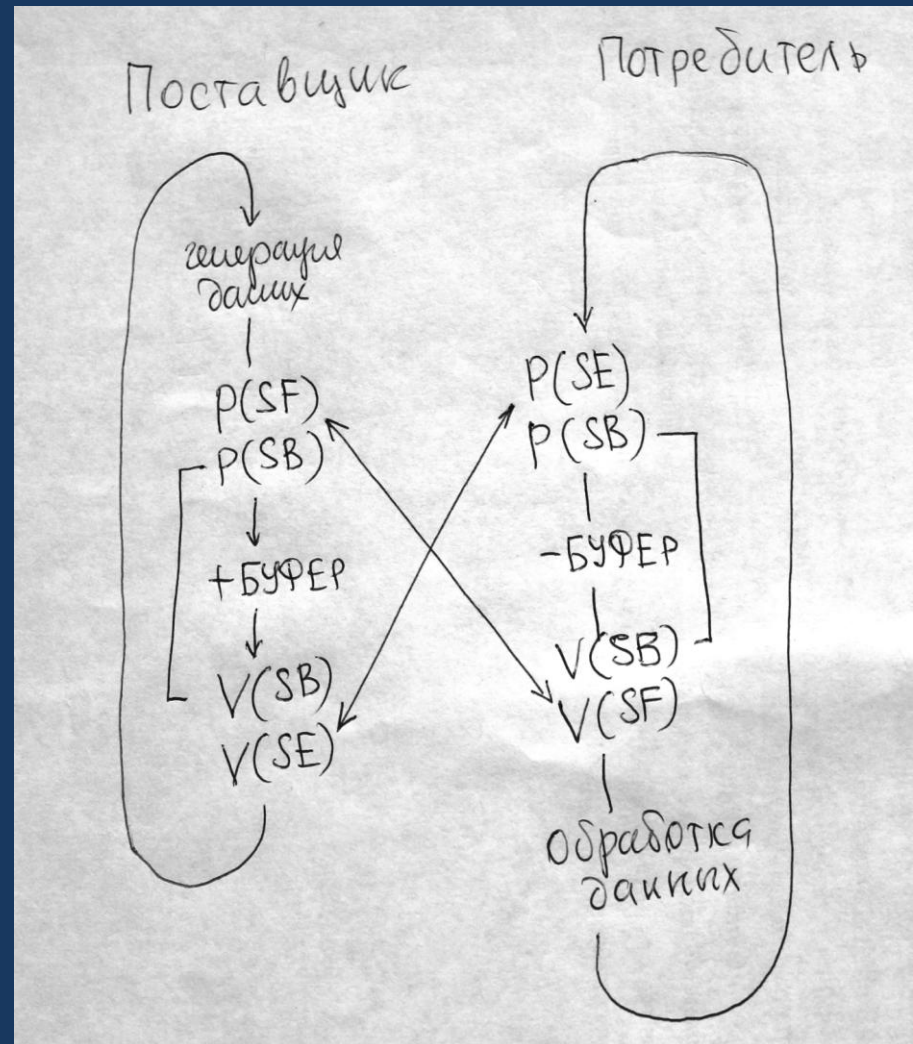




Семафоры. Поставщик-потребитель

Поставщик и потребитель (Producer, Consumer) — потоки, передающие данные через буферный пул размерностью N :

- $SE(0)$ — семафор «буфер пуст»
- $SF(N)$ - семафор «буфер полон»
- $SB(1)$ — семафор операции над буфером





Windows. Мониторинг процессов (ЦП)

CTRL+ALT+DEL — диспетчер задач

Диспетчер задач Windows

Файл Параметры Вид Справка

Приложения Процессы Службы Быстродействие Сеть Пользователи

Имя образа	ID п...	Пользователь	ЦП	Время ЦП	Базовы...	Дескрипторы	Счетчик потоков	Объекты USER	Объекты GDI	Описание
Бездействие ...	0	система	81	0:08:00	Н/Д	0	1	0	0	Процент времени безде
YandexDisk.exe	2976	romanow	14	0:01:29	Средний	542	42	61	114	Яндекс.Диск
avp.exe *32	1556	система	02	0:00:40	Средний	2 625	183	0	0	Kaspersky Anti-Virus
taskmgr.exe	1244	romanow	02	0:00:18	Высокий	117	7	105	126	Диспетчер задач Windo
avpui.exe *32	2304	romanow	02	0:00:14	Средний	727	19	51	64	Kaspersky Anti-Virus
System	4	система	00	0:00:11	Средний	2 407	131	0	0	NT Kernel & System
explorer.exe	2424	romanow	00	0:00:09	Средний	644	22	212	372	Проводник
svchost.exe	492	система	00	0:00:06	Средний	740	32	0	0	Хост-процесс для служб
svchost.exe	1516	система	00	0:00:04	Средний	322	13	0	0	Хост-процесс для служб
svchost.exe	584	система	00	0:00:03	Средний	1 262	39	0	0	Хост-процесс для служб
csrss.exe	624	система	00	0:00:02	Средний	327	10	78	174	Процесс исполнения кли
lsass.exe	688	система	00	0:00:01	Средний	751	7	0	0	Local Security Authority P
services.exe	656	система	00	0:00:01	Средний	209	8	0	0	Приложение служб и ко
svchost.exe	940	LOCAL SERVICE	00	0:00:01	Средний	569	23	0	0	Хост-процесс для служб
svchost.exe	828	система	00	0:00:01	Средний	358	10	0	0	Хост-процесс для служб
ksde.exe *32	2092	система	00	0:00:01	Средний	533	35	0	0	Kaspersky Secure Connec
svchost.exe	1396	LOCAL SERVICE	00	0:00:01	Средний	314	18	0	0	Хост-процесс для служб

☒ Отображать процессы всех пользователей

Завершить процесс

Процессов: 50 Загрузка ЦП: 20% Физическая память: 56%



Windows. Монитор ресурсов

Монитор ресурсов

Файл Монитор Справка

Обзор ЦП Память Диск Сеть

Процессы 5% - использование ЦП 100% максимальной частоты

Образ	ИД п...	Описание	Состояние	Потоки	ЦП	Среднее для ЦП
soffice.bin	912	OpenOffice 4.1.5	Выполняется	15	0	51.12
perfmon.exe	2316	Монитор ресурсов и производит...	Выполняется	19	3	4.82
avp.exe	1556	Kaspersky Anti-Virus	Выполняется	185	0	3.07
svchost.exe (LocalSystemNet...	492	Хост-процесс для служб Windows	Выполняется	34	0	1.63
YandexDisk.exe	2976	Яндекс.Диск	Выполняется	42	0	1.33
csrss.exe	624	Процесс исполнения клиент-серв...	Выполняется	10	0	1.19
System	4	NT Kernel & System	Выполняется	136	0	0.84
Системные прерывания	-	Отложенные вызовы процедур и ...	Выполняется	-	0	0.41
taskmgr.exe	5080	Диспетчер задач Windows	Выполняется	12	0	0.38

Службы 2% - использование ЦП

Имя	ИД п...	Описание	Состояние	Группа	ЦП	Среднее для ЦП
AVP18.0.0	1556	Kaspersky Anti-Virus Service 18.0.0	Выполняется		2	3.27
SysMain	492	Superfetch	Выполняется	LocalSystemNe...	0	1.63
PlugPlay	828	Plug-and-Play	Выполняется	DcomLaunch	0	0.71
BFE	1396	Служба базовой фильтрации	Выполняется	LocalServiceNo...	0	0.43
WSearch	3424	Windows Search	Выполняется		0	0.15

Связанные дескрипторы

Поиск дескрипторов



Windows. Монитор ресурсов

Связанные дескрипторы			
Отфильтровано по: YandexDisk.exe			
Образ	ИД процесса	Тип	Имя дескриптора
YandexDisk.exe	2976	ALPC Port	\RPC Control\OLED96DDADD1AA5
YandexDisk.exe	2976	Desktop	\Default
YandexDisk.exe	2976	Directory	\Sessions\1\BaseNamedObjects
YandexDisk.exe	2976	Directory	\KnownDlls
YandexDisk.exe	2976	Event	\BaseNamedObjects\C::Users:roma
YandexDisk.exe	2976	Event	\BaseNamedObjects\C::Users:roma
YandexDisk.exe			

Образ	ИД процесса	Тип	Имя дескриптора
YandexDisk.exe	2976	File	C:\Windows\winsxs\amd64
YandexDisk.exe	2976	File	C:\Windows\winsxs\amd64
YandexDisk.exe	2976	File	C:\Windows\winsxs\amd64
YandexDisk.exe	2976	File	C:\Windows\System32
YandexDisk.exe	2976	Key	\REGISTRY\USER\S-1-5-21-2
YandexDisk.exe	2976	Key	\REGISTRY\MACHINE\SOFTV
YandexDisk.exe	2976	Key	\REGISTRY\USER\S-1-5-21-2
YandexDisk.exe	2976		

Образ	ИД процесса	Тип	Имя дескриптора
YandexDisk.exe	2976	Mutant	\Sessions\1\BaseNamedObjects\DD
YandexDisk.exe	2976	Mutant	\Sessions\1\BaseNamedObjects\DD
YandexDisk.exe	2976	Mutant	\Sessions\1\BaseNamedObjects__D
YandexDisk.exe	2976	Mutant	\Sessions\1\BaseNamedObjects__D
YandexDisk.exe	2976	Section	\Sessions\1\BaseNamedObjects\wi
YandexDisk.exe	2976	Section	\Sessions\1\BaseNamedObjects\C::
YandexDisk.exe	2976	Section	\Sessions\1\BaseNamedObjects\C::
YandexDisk.exe	2976	Section	\Sessions\1\BaseNamedObjects\C::